Untitled

Jack

2026-04-09

Data Cleaning

```
housing_data <- read.csv("~/Downloads/melb_data.csv")

housing_clean <- housing_data[
  !is.na(housing_data$Price) &
  !is.na(housing_data$Rooms) &
  !is.na(housing_data$Bathroom) &
  !is.na(housing_data$Car) &
  !is.na(housing_data$Landsize) &
  !is.na(housing_data$Regionname),
]
nrow(housing_clean)
```

```
## [1] 13518
```

```
summary(housing_clean$Price)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   85000  650000  901000 1074796 1328000 9000000
```

```
summary(housing_clean$log_price)
```

```
## Length  Class   Mode
##      0   NULL   NULL
```

```
sum(is.na(housing_clean$Price))
```

```
## [1] 0
```

```
sum(is.na(housing_clean$log_price))
```

```
## [1] 0
```

```
length(unique(housing_clean$Price))
```

```
## [1] 2200
```

```
length(unique(housing_clean$log_price))
```

```
## [1] 0
```

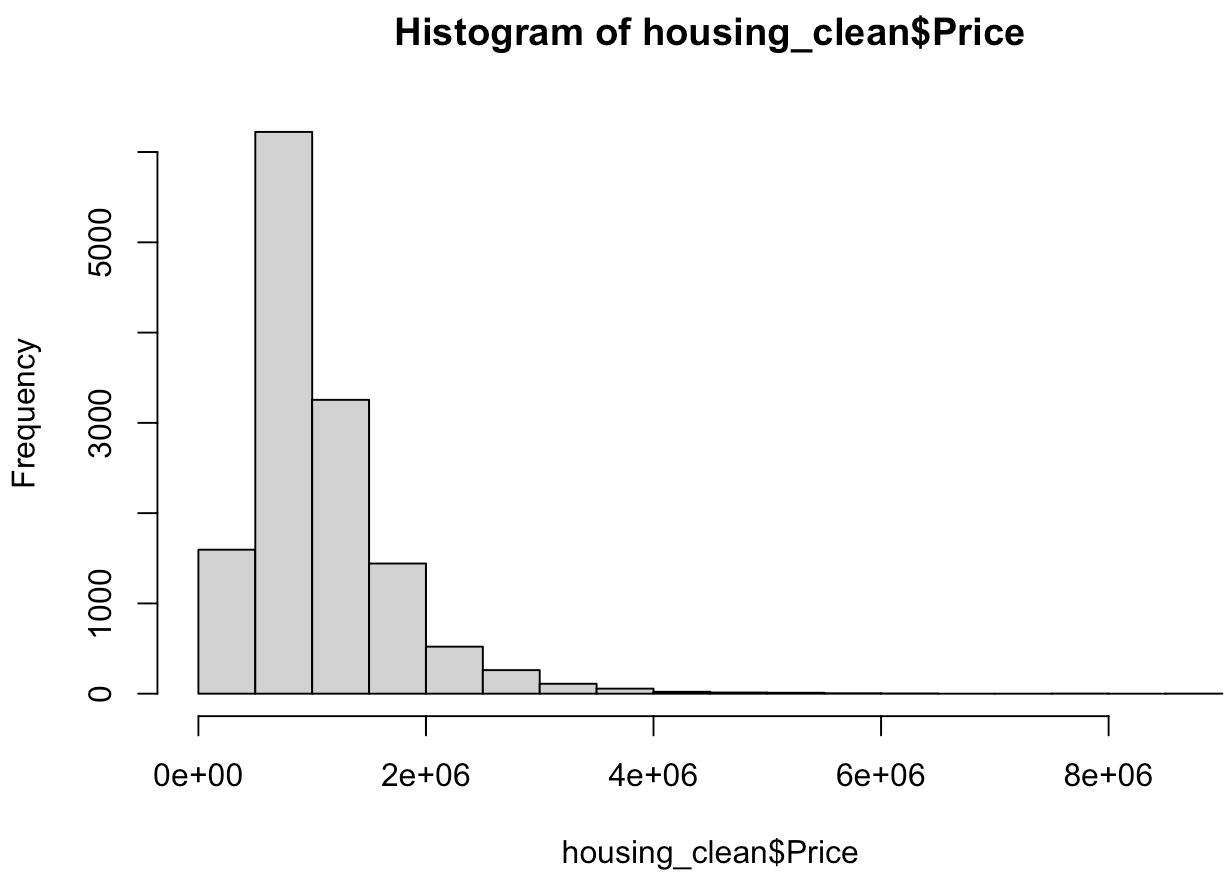
```
housing_clean$log_price <- log(housing_clean$Price)

summary(housing_clean$log_price)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   11.35   13.38   13.71   13.74   14.10   16.01
```

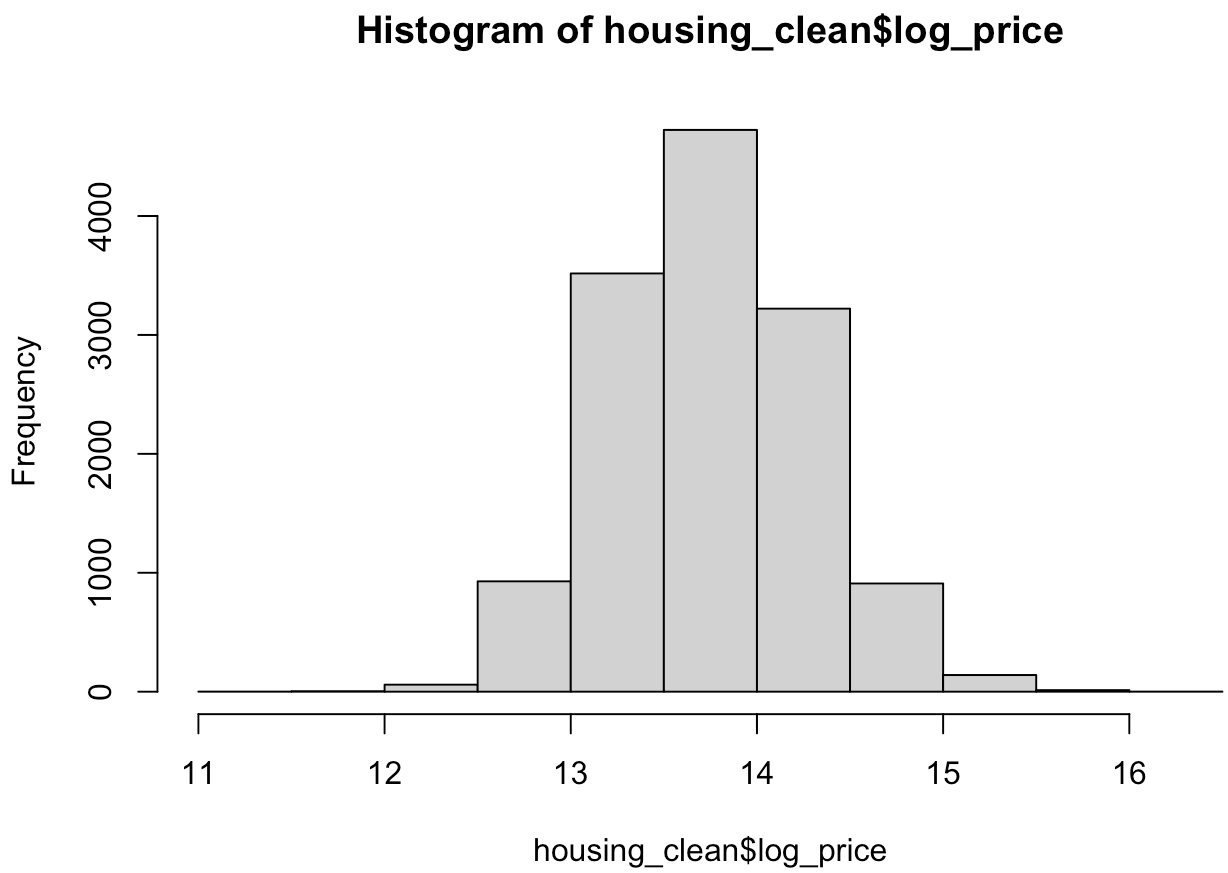
```
length(unique(housing_clean$log_price))
```

```
## [1] 2200
```

```
hist(housing_clean$Price)
```




```
hist(housing_clean$log_price)
```




```
table(housing_data$Bathroom, useNA = "ifany")
```

```
##
##    0    1    2    3    4    5    6    7    8
##   34 7512 4974  917  106   28    5    2    2
```

```
housing_data_model <- subset(housing_data, Bathroom >= 1)

housing_data_model$Bath_group <- ifelse(
  housing_data_model$Bathroom >= 3,
  "3plus",
  as.character(housing_data_model$Bathroom)
)

housing_data_model$Bath_group <- factor(
  housing_data_model$Bath_group,
  levels = c("1", "2", "3plus")
)

housing_data_model$Bath2 <- as.integer(housing_data_model$Bath_group == "2")
housing_data_model$Bath3plus <- as.integer(housing_data_model$Bath_group == "3plus")

table(housing_data_model$Bath_group, useNA = "ifany")
```

```
##
##     1     2 3plus
##  7512  4974  1060
```

```
summary(housing_data_model$Bathroom)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   1.000   1.000   1.538   2.000   8.000
```

```
library(cmdstanr)
```

```
## This is cmdstanr version 0.9.0
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: /Users/jackyip/.cmdstan/cmdstan-2.38.0
```

```
## - CmdStan version: 2.38.0
```

```
housing_data_model$log_price <- log(housing_data_model$Price)

housing_data_model$Date <- as.Date(housing_data_model$Date, format = "%d/%m/%Y")
housing_data_model$YearSold <- as.integer(format(housing_data_model$Date, "%Y"))
housing_data_model$Age <- housing_data_model$YearSold - housing_data_model$YearBuilt

model2_data <- subset(
  housing_data_model,
  !is.na(log_price) &
  !is.na(Distance) &
  !is.na(BuildingArea) &
  !is.na(Age) &
  !is.na(Bath2) &
  !is.na(Bath3plus) &
  !is.na(Regionname)
)

model2_data$region_id <- as.integer(factor(model2_data$Regionname))

data_list <- list(
  N = nrow(model2_data),
  R = length(unique(model2_data$region_id)),
  y = model2_data$log_price,
  distance = model2_data$Distance,
  buildingarea = model2_data$BuildingArea,
  age = model2_data$Age,
  bath2 = model2_data$Bath2,
  bath3plus = model2_data$Bath3plus,
  region = model2_data$region_id
)


mod <- cmdstan_model("~/Part2_Model2_Hierarchical.stan")
fit <- mod$sample(
  data = data_list,
  chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123
)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 251.5 seconds.
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 244.9 seconds.
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 3 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 3
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 3 Exception: normal_lpdf: Location parameter[1] is -inf, but must be finite! (i
n '/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan',
line 36, column 2 to line 39, column 4)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 3
```

```
## Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 242.7 seconds.
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 273.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 253.2 seconds.
## Total execution time: 1012.9 seconds.
```

```
## Warning: 12 of 4000 (0.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.
```

```
fit
```

```
##  variable    mean  median   sd  mad      q5     q95 rhat ess_bulk ess_tail
##  lp__     2670.65 2670.93 3.14 3.00 2665.06 2675.30 1.00      679     1437
##  beta0      13.60   13.60 0.09 0.08   13.46   13.74 1.01      806     1114
##  beta1      -0.03   -0.03 0.00 0.00   -0.03   -0.03 1.00     4337     3542
##  beta2       0.00    0.00 0.00 0.00    0.00    0.00 1.00     4195     3273
##  u_raw[1]    0.48    0.47 0.38 0.37   -0.13    1.11 1.01      799     1368
##  u_raw[2]    0.73    0.72 0.49 0.47   -0.04    1.57 1.00     1165     1921
##  u_raw[3]   -0.90   -0.89 0.46 0.46   -1.67   -0.16 1.00      751     1113
##  u_raw[4]    0.15    0.14 0.48 0.47   -0.61    0.95 1.00     1313     1749
##  u_raw[5]    1.06    1.04 0.47 0.48    0.31    1.84 1.00      838     1398
##  u_raw[6]    0.62    0.60 0.40 0.39   -0.01    1.28 1.01      809     1272
##
##  # showing 10 of 13738 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' op
tion)
```

```
fit$summary()
```

```
## # A tibble: 13,738 × 10
##    variable       mean   median      sd     mad       q5      q95  rhat ess_bulk
##    <chr>         <dbl>    <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
##  1 lp__     2671.       2.67e+3 3.14e+0 3.00e+0  2.67e+3  2.68e+3  1.00     679.
##  2 beta0      13.6      1.36e+1 8.75e-2 7.85e-2  1.35e+1  1.37e+1  1.01     807.
##  3 beta1      -0.0281  -2.81e-2 1.10e-3 1.10e-3 -2.99e-2 -2.63e-2  1.00    4338.
##  4 beta2       0.00323  3.23e-3 5.64e-5 5.57e-5  3.13e-3  3.32e-3  1.00    4195.
##  5 u_raw[1]    0.484    4.71e-1 3.80e-1 3.71e-1 -1.30e-1  1.11e+0  1.01     800.
##  6 u_raw[2]    0.732    7.15e-1 4.88e-1 4.73e-1 -3.69e-2  1.57e+0  1.00    1165.
##  7 u_raw[3]   -0.901   -8.91e-1 4.61e-1 4.65e-1 -1.67e+0 -1.60e-1  1.00     751.
##  8 u_raw[4]    0.150    1.44e-1 4.76e-1 4.72e-1 -6.15e-1  9.50e-1  1.00    1313.
##  9 u_raw[5]    1.06     1.04e+0 4.70e-1 4.80e-1  3.08e-1  1.84e+0  1.00     838.
## 10 u_raw[6]    0.616    6.03e-1 3.97e-1 3.91e-1 -1.35e-2  1.28e+0  1.01     809.
## # ℹ 13,728 more rows
## # ℹ 1 more variable: ess_tail <dbl>
```

```
fit$summary(c("beta0", "beta1", "beta2", "sigma", "tau"))
```

```
## # A tibble: 5 × 10
##   variable     mean   median        sd      mad       q5      q95  rhat ess_bulk
##   <chr>       <dbl>    <dbl>     <dbl>    <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.6     13.6     0.0875     7.85e-2 13.5     13.7      1.01     807.
## 2 beta1    -0.0281  -0.0281  0.00110    1.10e-3 -0.0299  -0.0263   1.00    4338.
## 3 beta2     0.00323  0.00323 0.0000564  5.57e-5  0.00313  0.00332  1.00    4195.
## 4 sigma     0.411    0.411   0.00352    3.50e-3  0.405    0.416    1.00    2224.
## 5 tau       0.234    0.216   0.0834     6.75e-2  0.136    0.390    1.00     799.
## # ℹ 1 more variable: ess_tail <dbl>
```

For the baseline model, we study how log house price is associated with distance from the CBD and land size. We standardize the continuous predictors before fitting so that coefficient magnitudes are easier to compare and posterior computation is more stable. Weakly informative normal priors are placed on the regression coefficients, and an exponential prior is placed on the residual standard deviation.

```
library(posterior)
```

```
## This is posterior version 1.7.0
```

```
##
## Attaching package: 'posterior'
```

```
## The following objects are masked from 'package:stats':
##
##     mad, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     %in%, match
```

```
draws <- as_draws_df(fit$draws(c("beta0", "beta1", "beta2", "sigma", "tau")))
head(draws)
```

```
## # A draws_df: 6 iterations, 1 chains, and 5 variables
##   beta0  beta1  beta2 sigma  tau
## 1    14 -0.029 0.0033  0.41 0.19
## 2    14 -0.027 0.0033  0.41 0.16
## 3    14 -0.028 0.0032  0.41 0.20
## 4    14 -0.028 0.0032  0.41 0.20
## 5    14 -0.028 0.0032  0.41 0.26
## 6    14 -0.029 0.0033  0.41 0.18
## # ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

Part 1 model

```
part1_data <- housing_data

part1_data <- subset(
  part1_data,
  !is.na(Price) &
  !is.na(Distance) &
  !is.na(BuildingArea) &
  Price > 0 &
  BuildingArea > 0
)

part1_data$log_price <- log(part1_data$Price)

summary(part1_data[, c("Price", "log_price", "Distance", "BuildingArea")])
```

```
##      Price           log_price        Distance      BuildingArea
##  Min.   : 131000   Min.   :11.78   Min.   : 0.00   Min.   :    1.0
##  1st Qu.: 630000   1st Qu.:13.35   1st Qu.: 6.10   1st Qu.:   93.0
##  Median : 895000   Median :13.70   Median : 9.20   Median :  126.0
##  Mean   :1079041   Mean   :13.74   Mean   :10.14   Mean   :  152.3
##  3rd Qu.:1335000   3rd Qu.:14.10   3rd Qu.:13.00   3rd Qu.:  174.0
##  Max.   :9000000   Max.   :16.01   Max.   :48.10   Max.   :44515.0
```

```
nrow(part1_data)
```

```
## [1] 7113
```

```
library(cmdstanr)

data_list_part1 <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  distance = part1_data$Distance,
  buildingarea = part1_data$BuildingArea
)

mod_part1 <- cmdstan_model("~/Model1.stan")

fit_part1 <- mod_part1$sample(
  data = data_list_part1,
  chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  refresh = 0
)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 finished in 75.2 seconds.
## Chain 2 finished in 127.8 seconds.
## Chain 3 finished in 52.4 seconds.
## Chain 4 finished in 57.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 78.3 seconds.
## Total execution time: 313.5 seconds.
```

For the baseline model, we study how log house price is associated with distance from the CBD and building area. Let $y_i = \log(\text{Price}_i)$ denote the response for house i. We use the model

Diagnostics: for trace plot + histogram

```
fit_part1$summary(c("beta0", "beta1", "beta2", "sigma"))
```

```
## # A tibble: 4 × 10
##   variable      mean    median       sd     mad       q5      q95  rhat ess_bulk
##   <chr>        <dbl>     <dbl>    <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.9      13.9       1.20e-2 1.17e-2  1.39e+1  1.39e+1  1.01     682.
## 2 beta1    -0.0157   -0.0157    1.02e-3 1.03e-3 -1.74e-2 -1.41e-2  1.00     913.
## 3 beta2     0.000114  0.000114  1.18e-5 1.21e-5  9.46e-5  1.33e-4  1.00    3928.
## 4 sigma     0.531     0.531     4.29e-3 4.28e-3  5.24e-1  5.38e-1  1.00    2473.
## # ℹ 1 more variable: ess_tail <dbl>
```

```
library(bayesplot)
```

```
## This is bayesplot version 1.15.0
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

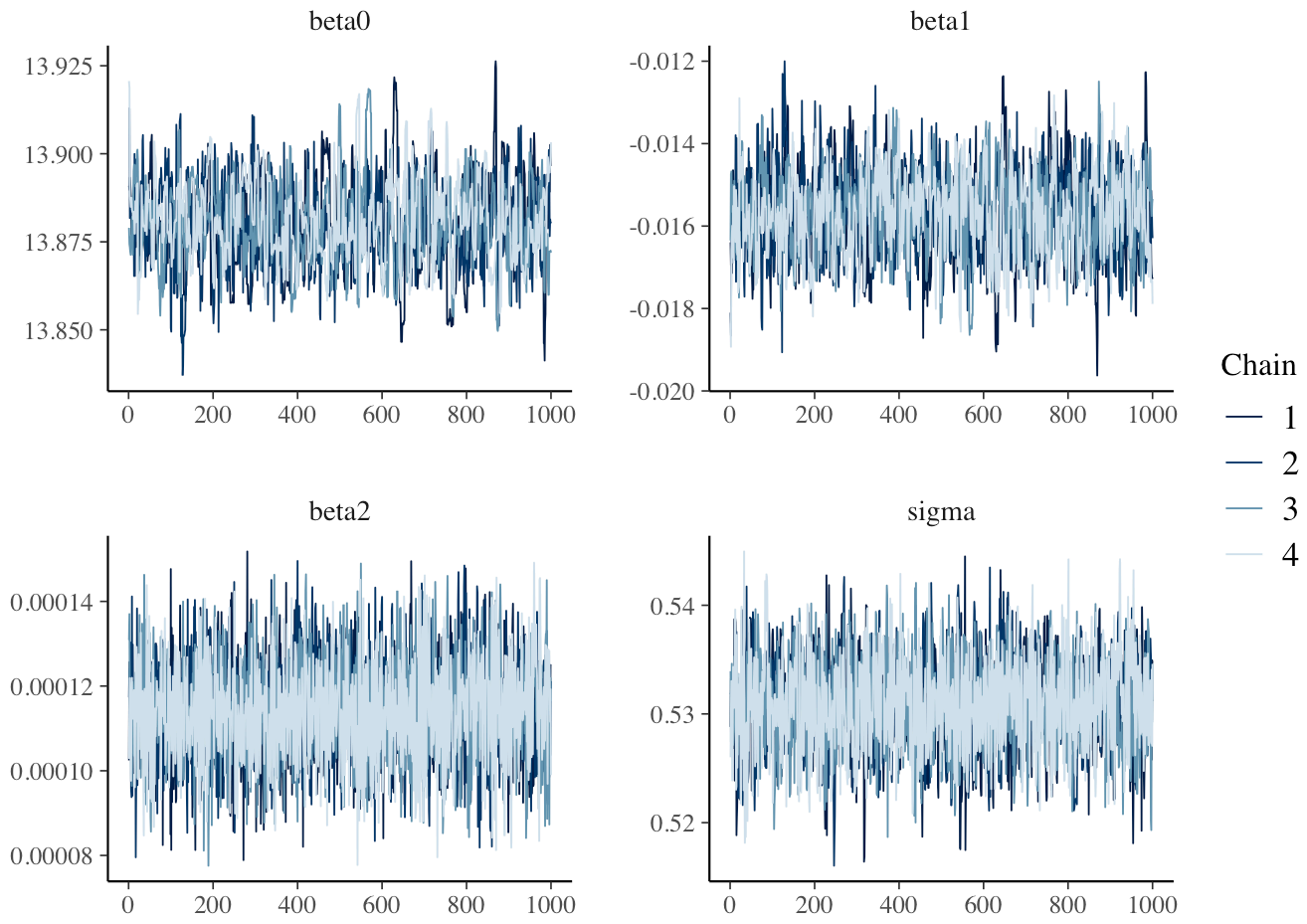
```
##    * Does _not_ affect other ggplot2 plots
```

```
##    * See ?bayesplot_theme_set for details on theme setting
```

```
##
## Attaching package: 'bayesplot'
```

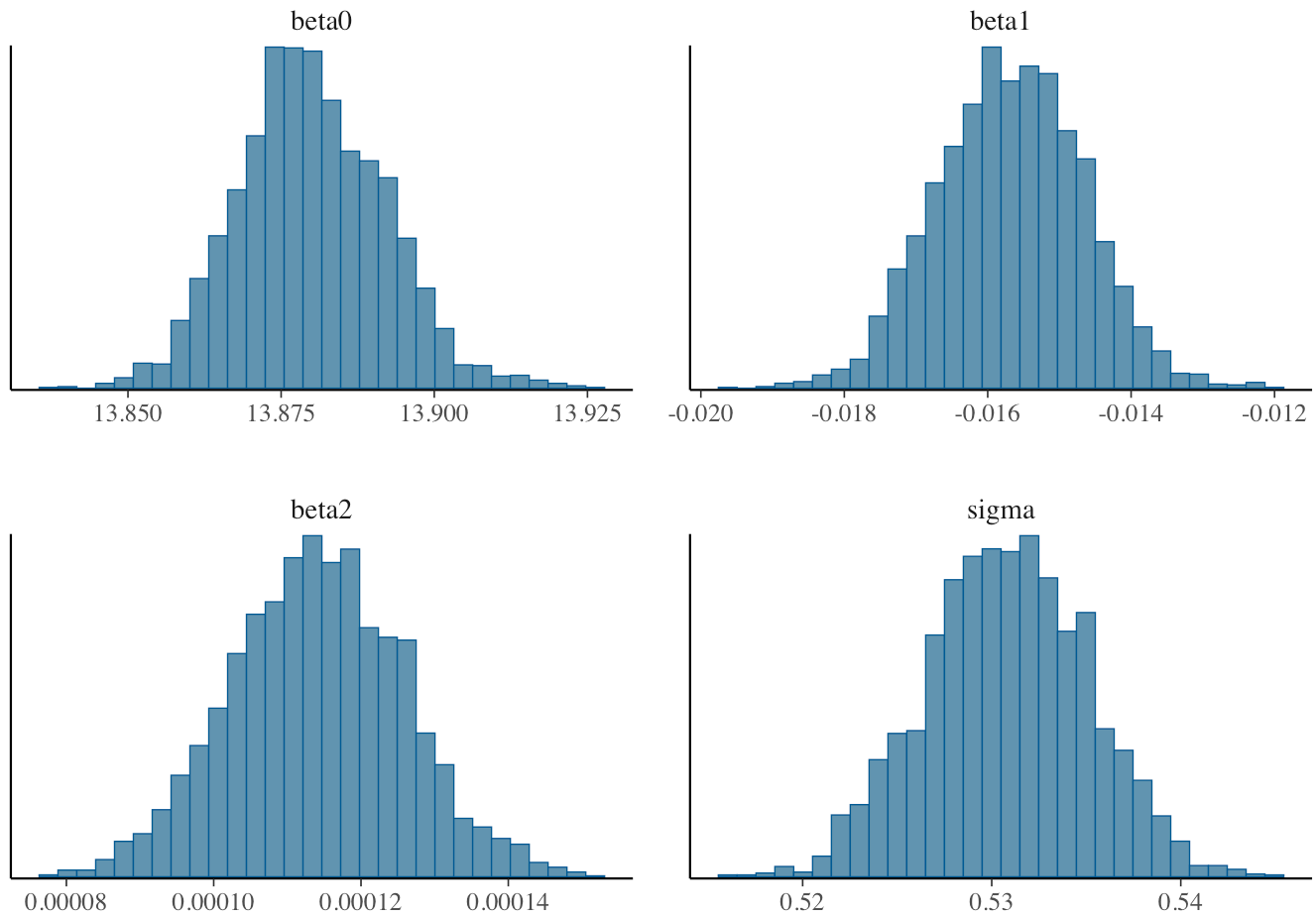
```
## The following object is masked from 'package:posterior':
##
##     rhat
```

```
mcmc_trace(fit_part1$draws(c("beta0", "beta1", "beta2", "sigma")))
```




```
mcmc_hist(fit_part1$draws(c("beta0", "beta1", "beta2", "sigma")))
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



the chains are mixing well the posterior is unimodal and smooth Stan is giving a stable benchmark fit

custom MH for Part 1

```
log_posterior_part1 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]

  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  # log-priors
  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) +
    eta  # Jacobian adjustment for sigma = exp(eta)

  # log-likelihood
  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

Random walk MH sampler

```
mh_step <- function(current, current_lp, proposal_sds, y, distance, buildingarea) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- log_posterior_part1(proposal, y, distance, buildingarea)

  log_alpha <- proposal_lp - current_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}
```

Kernel-mixture sampler

```
mh_mixture_part1 <- function(y, distance, buildingarea,
                             init,
                             n_iter = 20000,
                             proposal_sds_small = c(0.01, 0.0003, 0.000003, 0.005),
                             proposal_sds_large = c(0.03, 0.0010, 0.000010, 0.015),
                             mix_prob_small = 0.5) {

  stopifnot(length(init) == 4)

  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  accepted <- logical(n_iter)
  kernel_used <- integer(n_iter)

  current <- init
  current_lp <- log_posterior_part1(current, y, distance, buildingarea)

  for (i in seq_len(n_iter)) {
    use_small <- runif(1) < mix_prob_small

    if (use_small) {
      kernel_used[i] <- 1
      out <- mh_step(
        current = current,
        current_lp = current_lp,
        proposal_sds = proposal_sds_small,
        y = y,
        distance = distance,
        buildingarea = buildingarea
      )
    } else {
      kernel_used[i] <- 2
      out <- mh_step(
        current = current,
        current_lp = current_lp,
        proposal_sds = proposal_sds_large,
        y = y,
        distance = distance,
        buildingarea = buildingarea
      )
    }

    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(
```

```
    samples = samples,
    accepted = accepted,
    kernel_used = kernel_used,
    acceptance_rate = mean(accepted),
    acceptance_rate_small = mean(accepted[kernel_used == 1]),
    acceptance_rate_large = mean(accepted[kernel_used == 2])
  )
}
```

```
y <- part1_data$log_price
distance <- part1_data$Distance
buildingarea <- part1_data$BuildingArea

init <- c(13.88, -0.0157, 0.000114, log(0.53))

mh_fit <- mh_mixture_part1(
  y = y,
  distance = distance,
  buildingarea = buildingarea,
  init = init,
  n_iter = 30000
)

mh_fit$acceptance_rate
```

```
## [1] 0.2983333
```

```
mh_fit$acceptance_rate_small
```

```
## [1] 0.4849299
```

```
mh_fit$acceptance_rate_large
```

```
## [1] 0.1142819
```

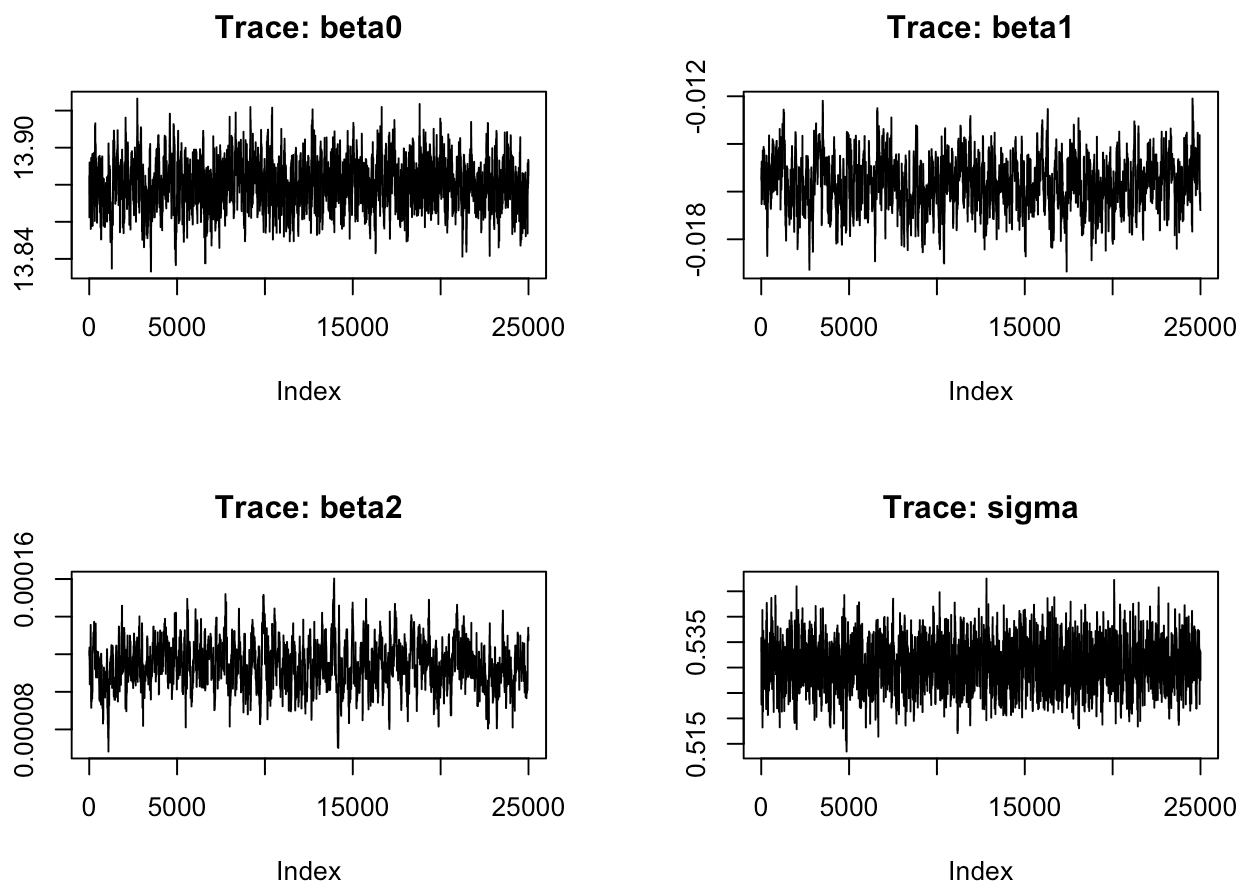
Burn-in and transform back to sigma

```
burn_in <- 5000

mh_samples <- as.data.frame(mh_fit$samples[(burn_in + 1):nrow(mh_fit$samples), ])
mh_samples$sigma <- exp(mh_samples$eta)

summary(mh_samples[, c("beta0", "beta1", "beta2", "sigma")])
```

```
##      beta0           beta1              beta2               sigma
##  Min.   :13.83   Min.   :-0.01935   Min.   :6.824e-05   Min.   :0.5135
##  1st Qu.:13.87   1st Qu.:-0.01639   1st Qu.:1.065e-04   1st Qu.:0.5277
##  Median :13.88   Median :-0.01564   Median :1.139e-04   Median :0.5307
##  Mean   :13.88   Mean   :-0.01566   Mean   :1.140e-04   Mean   :0.5307
##  3rd Qu.:13.89   3rd Qu.:-0.01493   3rd Qu.:1.216e-04   3rd Qu.:0.5337
##  Max.   :13.93   Max.   :-0.01210   Max.   :1.602e-04   Max.   :0.5475
```

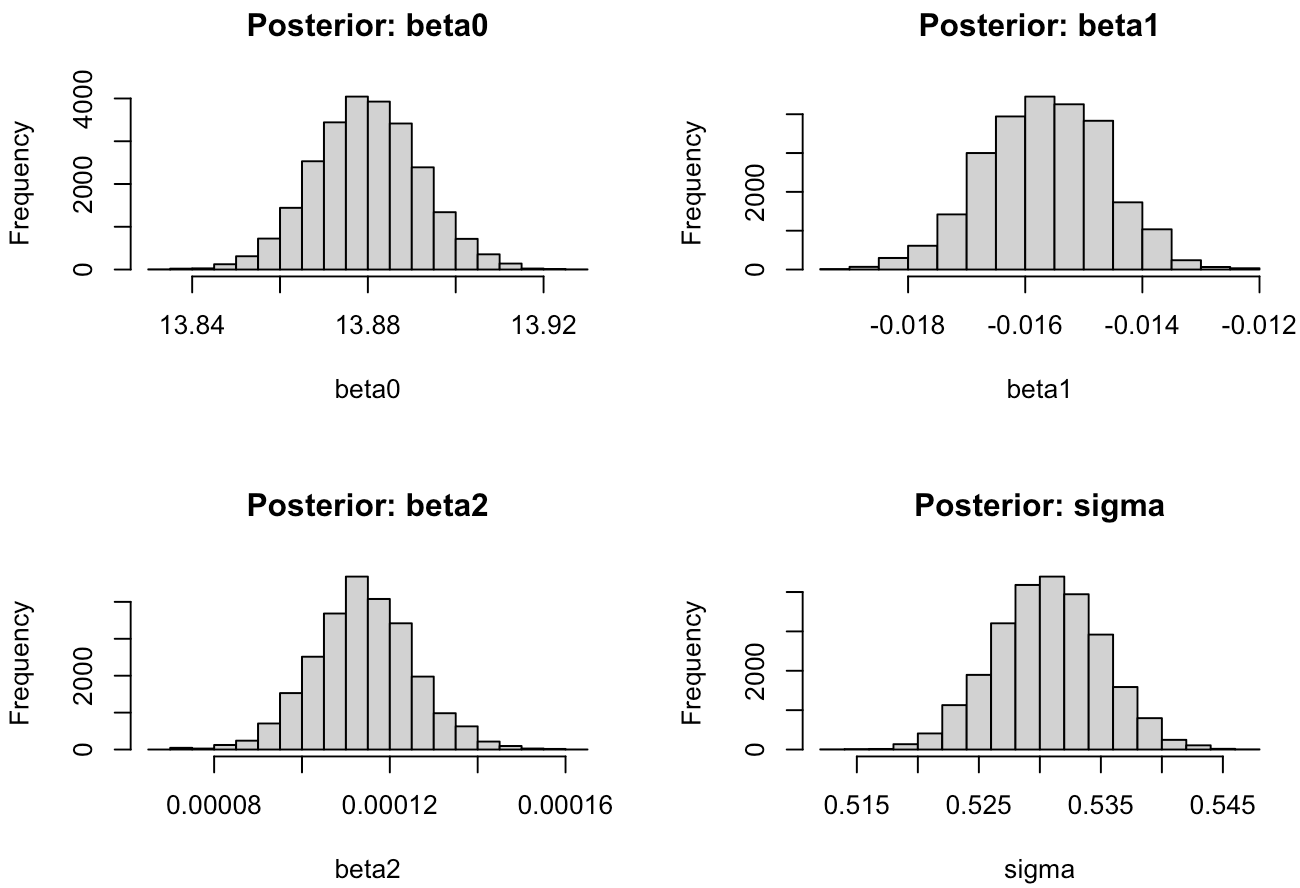
Trace plots

```
par(mfrow = c(2, 2))
plot(mh_samples$beta0, type = "l", main = "Trace: beta0", ylab = "")
plot(mh_samples$beta1, type = "l", main = "Trace: beta1", ylab = "")
plot(mh_samples$beta2, type = "l", main = "Trace: beta2", ylab = "")
plot(mh_samples$sigma, type = "l", main = "Trace: sigma", ylab = "")
```



Histograms

```
par(mfrow = c(2, 2))
hist(mh_samples$beta0, main = "Posterior: beta0", xlab = "beta0")
hist(mh_samples$beta1, main = "Posterior: beta1", xlab = "beta1")
hist(mh_samples$beta2, main = "Posterior: beta2", xlab = "beta2")
hist(mh_samples$sigma, main = "Posterior: sigma", xlab = "sigma")
```



Compare with Stan

```
stan_summary <- fit_part1$summary(c("beta0", "beta1", "beta2", "sigma"))
stan_summary[, c("variable", "mean", "median", "sd", "q5", "q95", "rhat")]
```

```
## # A tibble: 4 × 7
##   variable      mean    median        sd         q5       q95  rhat
##   <chr>        <dbl>     <dbl>     <dbl>      <dbl>     <dbl> <dbl>
## 1 beta0    13.9      13.9      0.0120    13.9       13.9       1.01
## 2 beta1    -0.0157   -0.0157   0.00102   -0.0174    -0.0141    1.00
## 3 beta2     0.000114  0.000114 0.0000118  0.0000946  0.000133  1.00
## 4 sigma     0.531     0.531    0.00429    0.524      0.538     1.00
```

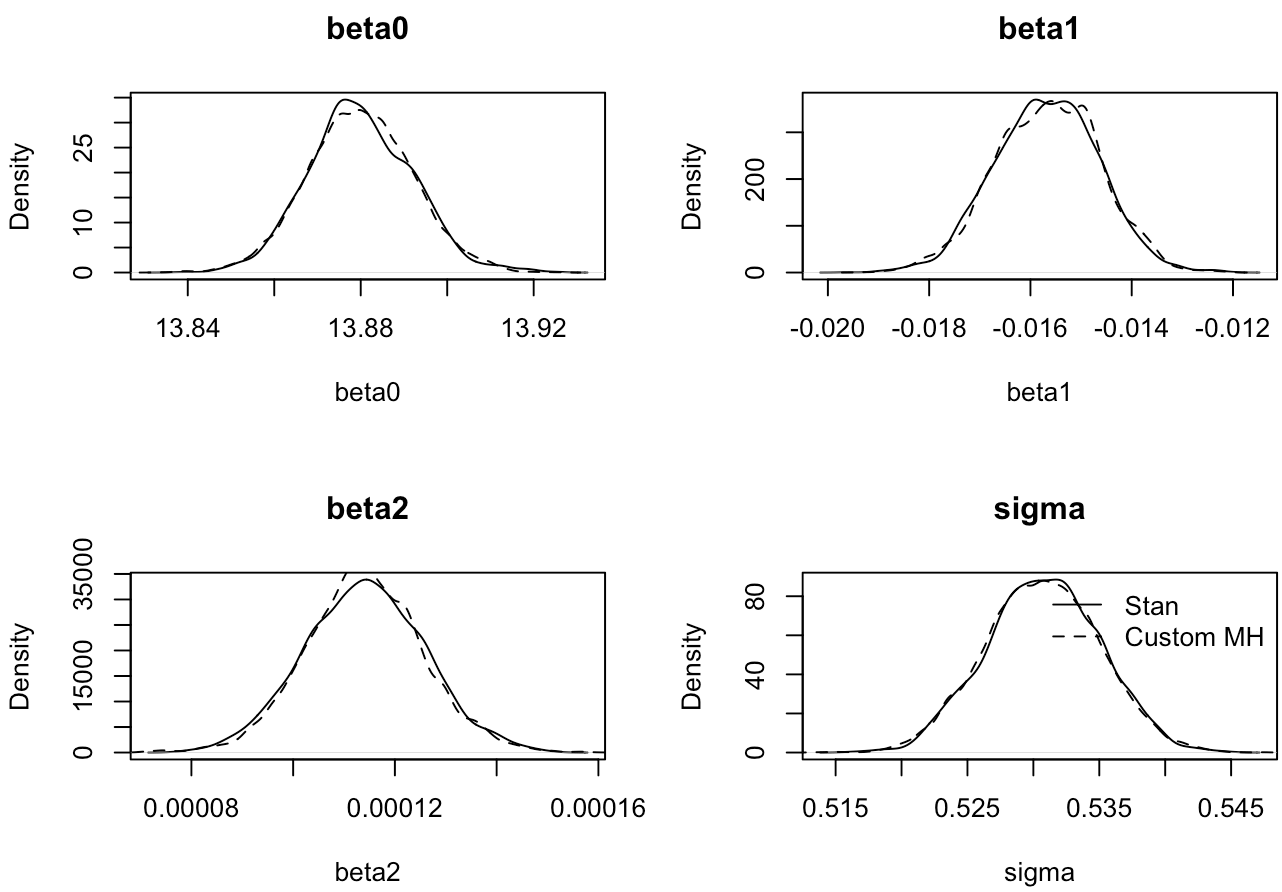
MH

```
mh_compare <- data.frame(
  variable = c("beta0", "beta1", "beta2", "sigma"),
  mean = c(mean(mh_samples$beta0),
           mean(mh_samples$beta1),
           mean(mh_samples$beta2),
           mean(mh_samples$sigma)),
  median = c(median(mh_samples$beta0),
             median(mh_samples$beta1),
             median(mh_samples$beta2),
             median(mh_samples$sigma)),
  sd = c(sd(mh_samples$beta0),
         sd(mh_samples$beta1),
         sd(mh_samples$beta2),
         sd(mh_samples$sigma)),
  q5 = c(quantile(mh_samples$beta0, 0.05),
         quantile(mh_samples$beta1, 0.05),
         quantile(mh_samples$beta2, 0.05),
         quantile(mh_samples$sigma, 0.05)),
  q95 = c(quantile(mh_samples$beta0, 0.95),
          quantile(mh_samples$beta1, 0.95),
          quantile(mh_samples$beta2, 0.95),
          quantile(mh_samples$sigma, 0.95))
)

mh_compare
```

```
##   variable          mean        median           sd            q5           q95
## 1    beta0 13.8798959416 13.8797557503 0.0120882223  1.386028e+01 13.9000210960
## 2    beta1 -0.0156593513 -0.0156372017 0.0010377056 -1.737841e-02 -0.0139506173
## 3    beta2  0.0001140166  0.0001139299 0.0000115166  9.548867e-05  0.0001333179
## 4    sigma  0.5306736508  0.5306760841 0.0043702920  5.234585e-01  0.5378728386
```

Overlay plots against Stan

```
library(posterior)

stan_draws <- as_draws_df(fit_part1$draws(c("beta0", "beta1", "beta2", "sigma")))

par(mfrow = c(2, 2))

plot(density(stan_draws$beta0), main = "beta0", xlab = "beta0")
lines(density(mh_samples$beta0), lty = 2)

plot(density(stan_draws$beta1), main = "beta1", xlab = "beta1")
lines(density(mh_samples$beta1), lty = 2)

plot(density(stan_draws$beta2), main = "beta2", xlab = "beta2")
lines(density(mh_samples$beta2), lty = 2)

plot(density(stan_draws$sigma), main = "sigma", xlab = "sigma")
lines(density(mh_samples$sigma), lty = 2)

legend("topright", legend = c("Stan", "Custom MH"), lty = c(1, 2), bty = "n")
```



```
part1_design <- subset(
  housing_data,
  !is.na(Distance) &
  !is.na(BuildingArea) &
  Distance > 0 &
  BuildingArea > 0,
  select = c(Distance, BuildingArea)
)
```

Forward simulator

```
forward_part1_dataset <- function(design_df) {
  beta0 <- rnorm(1, mean = 0, sd = 10)
  beta1 <- rnorm(1, mean = 0, sd = 1)
  beta2 <- rnorm(1, mean = 0, sd = 1)
  sigma <- rexp(1, rate = 1)

  distance <- design_df$Distance
  buildingarea <- design_df$BuildingArea

  mu <- beta0 + beta1 * distance + beta2 * buildingarea
  log_price <- rnorm(length(mu), mean = mu, sd = sigma)

  list(
    data = data.frame(
      Distance = distance,
      BuildingArea = buildingarea,
      log_price = log_price
    ),
    theta = c(beta0 = beta0, beta1 = beta1, beta2 = beta2, eta = log(sigma))
  )
}
```

MH mixture sampler and return the final state

```
run_mh_from_forward <- function(design_df, n_iter = 2000) {
  sim <- forward_part1_dataset(design_df)

  fit <- mh_mixture_part1(
    y = sim$data$log_price,
    distance = sim$data$Distance,
    buildingarea = sim$data$BuildingArea,
    init = sim$theta,
    n_iter = n_iter
  )

  final_theta <- fit$samples[n_iter, ]

  list(
    initial_theta = sim$theta,
    final_theta = final_theta,
    acceptance_rate = fit$acceptance_rate
  )
}
```

Build the invariance check

```
set.seed(123)

n_rep <- 300
n_iter_check <- 2000

# optional: use a smaller subset for speed
idx <- sample(seq_len(nrow(part1_design)), size = 200, replace = FALSE)
part1_design_small <- part1_design[idx, ]

initial_mat <- matrix(NA_real_, nrow = n_rep, ncol = 4)
final_mat   <- matrix(NA_real_, nrow = n_rep, ncol = 4)
acc_rates   <- numeric(n_rep)

colnames(initial_mat) <- c("beta0", "beta1", "beta2", "eta")
colnames(final_mat)   <- c("beta0", "beta1", "beta2", "eta")

for (m in 1:n_rep) {
  out <- run_mh_from_forward(
    design_df = part1_design_small,
    n_iter = n_iter_check
  )

  initial_mat[m, ] <- out$initial_theta
  final_mat[m, ]   <- out$final_theta
  acc_rates[m]     <- out$acceptance_rate
}

initial_df <- as.data.frame(initial_mat)
final_df   <- as.data.frame(final_mat)

initial_df$sigma <- exp(initial_df$eta)
final_df$sigma   <- exp(final_df$eta)
```

KS tests

```
ks_beta0 <- ks.test(initial_df$beta0, final_df$beta0)
```

```
## Warning in ks.test.default(initial_df$beta0, final_df$beta0): p-value will be
## approximate in the presence of ties
```

```
ks_beta1 <- ks.test(initial_df$beta1, final_df$beta1)
```

```
## Warning in ks.test.default(initial_df$beta1, final_df$beta1): p-value will be
## approximate in the presence of ties
```

```
ks_beta2 <- ks.test(initial_df$beta2, final_df$beta2)
```

```
## Warning in ks.test.default(initial_df$beta2, final_df$beta2): p-value will be
## approximate in the presence of ties
```

```
ks_sigma <- ks.test(initial_df$sigma, final_df$sigma)
```

```
## Warning in ks.test.default(initial_df$sigma, final_df$sigma): p-value will be
## approximate in the presence of ties
```

```
ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_df$beta0 and final_df$beta0
## D = 0.013333, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_df$beta1 and final_df$beta1
## D = 0.013333, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_beta2
```

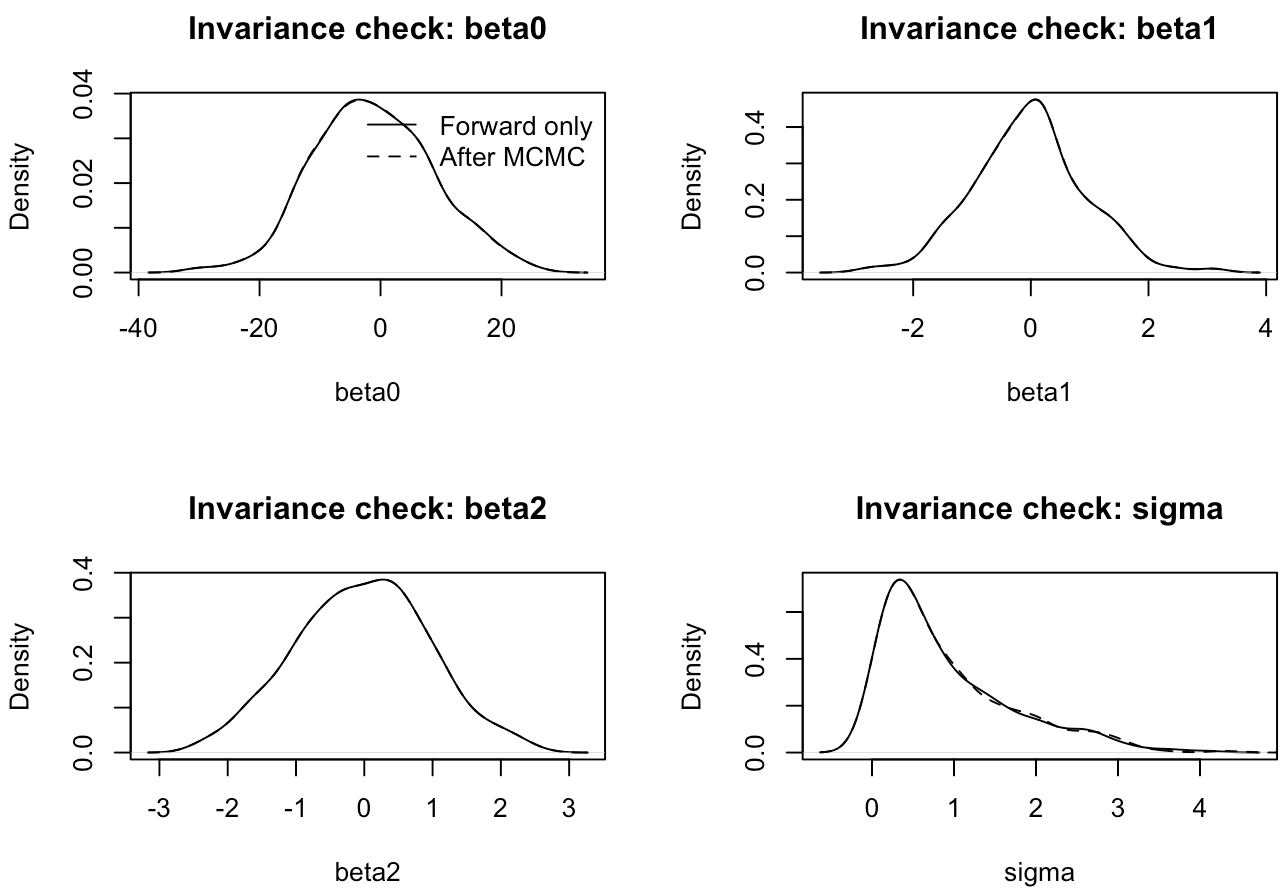
```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_df$beta2 and final_df$beta2
## D = 0.0066667, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  initial_df$sigma and final_df$sigma
## D = 0.023333, p-value = 1
## alternative hypothesis: two-sided
```

Visual comparison plots

```
par(mfrow = c(2, 2))

plot(density(initial_df$beta0), main = "Invariance check: beta0", xlab = "beta0")
lines(density(final_df$beta0), lty = 2)
legend("topright", legend = c("Forward only", "After MCMC"), lty = c(1, 2), bty = "n")

plot(density(initial_df$beta1), main = "Invariance check: beta1", xlab = "beta1")
lines(density(final_df$beta1), lty = 2)

plot(density(initial_df$beta2), main = "Invariance check: beta2", xlab = "beta2")
lines(density(final_df$beta2), lty = 2)

plot(density(initial_df$sigma), main = "Invariance check: sigma", xlab = "sigma")
lines(density(final_df$sigma), lty = 2)
```



Short summary table of means

```
rbind(
  forward_only = c(
    beta0 = mean(initial_df$beta0),
    beta1 = mean(initial_df$beta1),
    beta2 = mean(initial_df$beta2),
    sigma = mean(initial_df$sigma)
  ),
  after_mcmc = c(
    beta0 = mean(final_df$beta0),
    beta1 = mean(final_df$beta1),
    beta2 = mean(final_df$beta2),
    sigma = mean(final_df$sigma)
  )
)
```

```
##                  beta0       beta1       beta2     sigma
## forward_only -1.114042 -0.02739459 -0.02480827 0.9129531
## after_mcmc   -1.129360 -0.02684283 -0.02481409 0.9112632
```

Part 2 for Hierarchical

```
part2_data <- housing_data

part2_data <- subset(
  part2_data,
  !is.na(Price) &
  !is.na(Distance) &
  !is.na(BuildingArea) &
  !is.na(Regionname) &
  Price > 0 &
  BuildingArea > 0
)

part2_data$log_price <- log(part2_data$Price)

part2_data$region_id <- as.integer(factor(part2_data$Regionname))

summary(part2_data[, c("Price", "log_price", "Distance", "BuildingArea")])
```

```
##      Price           log_price        Distance      BuildingArea
##  Min.   : 131000   Min.   :11.78   Min.   : 0.00   Min.   :    1.0
##  1st Qu.: 630000   1st Qu.:13.35   1st Qu.: 6.10   1st Qu.:   93.0
##  Median : 895000   Median :13.70   Median : 9.20   Median :  126.0
##  Mean   :1079041   Mean   :13.74   Mean   :10.14   Mean   :  152.3
##  3rd Qu.:1335000   3rd Qu.:14.10   3rd Qu.:13.00   3rd Qu.:  174.0
##  Max.   :9000000   Max.   :16.01   Max.   :48.10   Max.   :44515.0
```

```
table(part2_data$Regionname)
```

```
##
##       Eastern Metropolitan           Eastern Victoria
##                        708                         30
##      Northern Metropolitan          Northern Victoria
##                       2090                         28
## South-Eastern Metropolitan      Southern Metropolitan
##                        220                       2409
##       Western Metropolitan           Western Victoria
##                       1607                         21
```

```
length(unique(part2_data$region_id))
```

```
## [1] 8
```

```
nrow(part2_data)
```

```
## [1] 7113
```

Data list

```
data_list_part2 <- list(
  N = nrow(part2_data),
  R = length(unique(part2_data$region_id)),
  y = part2_data$log_price,
  distance = part2_data$Distance,
  buildingarea = part2_data$BuildingArea,
  region = part2_data$region_id
)
```

```
library(cmdstanr)

mod_part2 <- cmdstan_model("~/Part2_Model2_Hierarchical.stan")

fit_part2 <- mod_part2$sample(
  data = data_list_part2,
  chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  refresh = 0
)
```

```
## Running MCMC with 4 sequential chains...
##
## Chain 1 finished in 420.9 seconds.
## Chain 2 finished in 434.1 seconds.
## Chain 3 finished in 403.3 seconds.
## Chain 4 finished in 443.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 425.5 seconds.
## Total execution time: 1702.4 seconds.
```

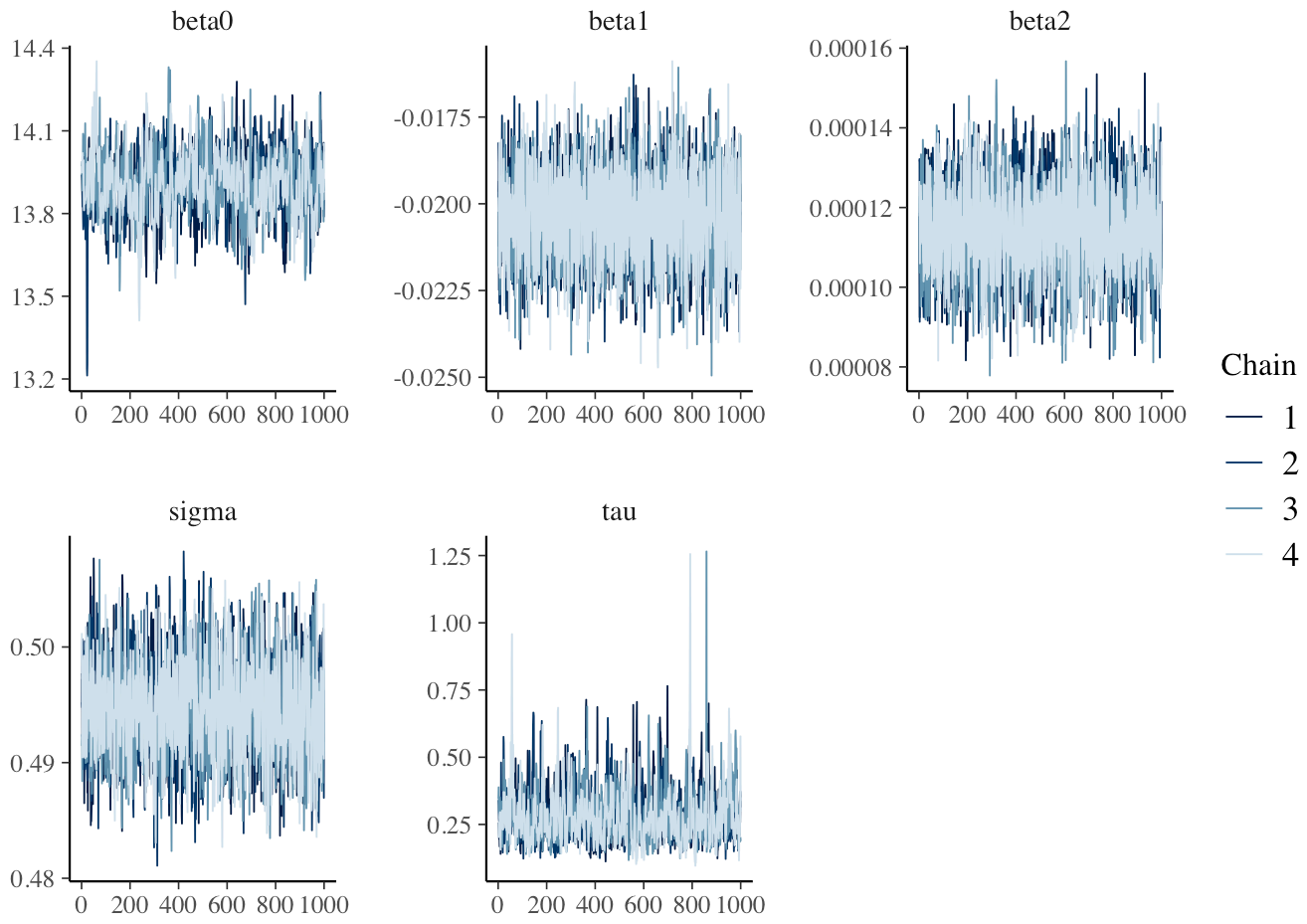
```
fit_part2$summary(c("beta0", "beta1", "beta2", "sigma", "tau"))
```

```
## # A tibble: 5 × 10
##   variable      mean    median       sd     mad       q5      q95  rhat ess_bulk
##   <chr>        <dbl>     <dbl>    <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.9      13.9       1.12e-1 1.01e-1  1.37e+1  1.41e+1  1.01     554.
## 2 beta1    -0.0204   -0.0204    1.28e-3 1.29e-3 -2.25e-2 -1.83e-2  1.00    4500.
## 3 beta2     0.000114  0.000114  1.12e-5 1.15e-5  9.58e-5  1.33e-4  1.00    3864.
## 4 sigma     0.495     0.495     4.07e-3 4.08e-3  4.88e-1  5.02e-1  1.00    2742.
## 5 tau       0.279     0.260     9.49e-2 7.67e-2  1.64e-1  4.52e-1  1.00     787.
## # ℹ 1 more variable: ess_tail <dbl>
```

```
fit_part2$summary("u")
```

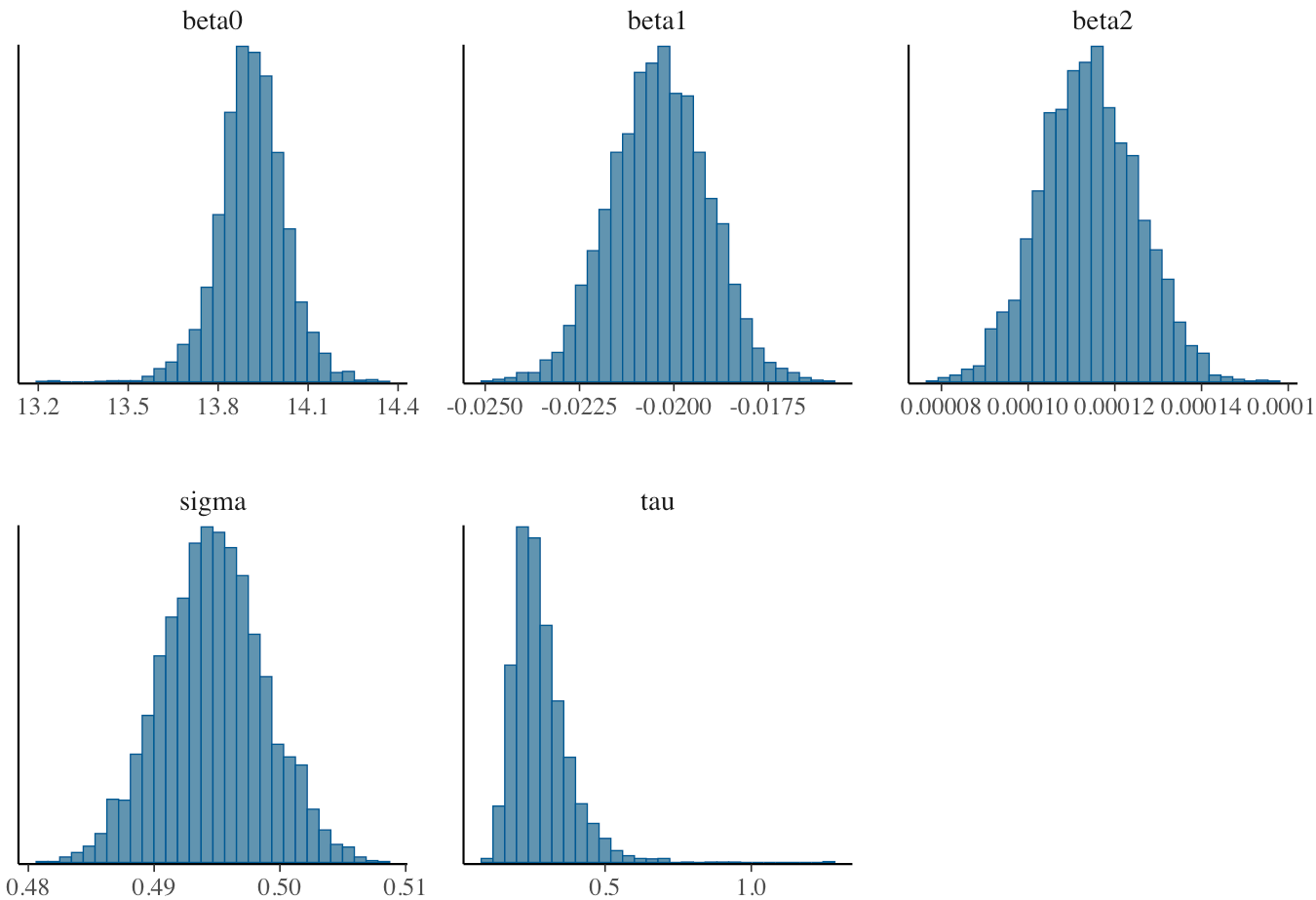
```
## # A tibble: 8 × 10
##   variable   mean median    sd    mad      q5      q95  rhat ess_bulk ess_tail
##   <chr>     <dbl>  <dbl> <dbl>  <dbl>   <dbl>    <dbl> <dbl>    <dbl>    <dbl>
## 1 u[1]      0.202  0.201 0.110 0.0973  0.0267  0.384    1.01     561.     989.
## 2 u[2]      0.159  0.156 0.131 0.127  -0.0493  0.372    1.00     823.    1556.
## 3 u[3]     -0.182 -0.182 0.110 0.0982 -0.357  -0.00266  1.01     551.     903.
## 4 u[4]     -0.148 -0.146 0.134 0.129  -0.364   0.0677   1.00     947.    1493.
## 5 u[5]      0.221  0.220 0.112 0.0988  0.0443  0.403    1.01     619.    1043.
## 6 u[6]      0.225  0.225 0.110 0.0972  0.0516  0.409    1.01     557.     909.
## 7 u[7]     -0.147 -0.147 0.110 0.0960 -0.321   0.0322   1.01     549.     933.
## 8 u[8]     -0.339 -0.335 0.143 0.140  -0.574  -0.108    1.01     974.    1584.
```

```
library(bayesplot)

mcmc_trace(fit_part2$draws(c("beta0", "beta1", "beta2", "sigma", "tau")))
```




```
mcmc_hist(fit_part2$draws(c("beta0", "beta1", "beta2", "sigma", "tau")))
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```
fit_part2$summary(c("beta0", "beta1", "beta2", "sigma", "tau"))[,
  c("variable", "mean", "median", "sd", "q5", "q95", "rhat", "ess_bulk", "ess_tail")]
```

```
## # A tibble: 5 × 9
##   variable      mean    median      sd       q5      q95  rhat ess_bulk ess_tail
##   <chr>        <dbl>     <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    13.9      13.9      1.12e-1  1.37e+1  1.41e+1  1.01     554.     920.
## 2 beta1    -0.0204   -0.0204   1.28e-3 -2.25e-2 -1.83e-2  1.00    4500.    3182.
## 3 beta2     0.000114  0.000114 1.12e-5  9.58e-5  1.33e-4  1.00    3864.    2864.
## 4 sigma     0.495     0.495    4.07e-3  4.88e-1  5.02e-1  1.00    2742.    2739.
## 5 tau       0.279     0.260    9.49e-2  1.64e-1  4.52e-1  1.00     787.    1233.
```

```
fit_part2$summary("u")
```

```
## # A tibble: 8 × 10
##   variable   mean median    sd    mad      q5      q95  rhat ess_bulk ess_tail
##   <chr>     <dbl>  <dbl> <dbl>  <dbl>   <dbl>    <dbl> <dbl>    <dbl>    <dbl>
## 1 u[1]      0.202  0.201 0.110 0.0973  0.0267  0.384    1.01     561.     989.
## 2 u[2]      0.159  0.156 0.131 0.127  -0.0493  0.372    1.00     823.    1556.
## 3 u[3]     -0.182 -0.182 0.110 0.0982 -0.357  -0.00266  1.01     551.     903.
## 4 u[4]     -0.148 -0.146 0.134 0.129  -0.364   0.0677   1.00     947.    1493.
## 5 u[5]      0.221  0.220 0.112 0.0988  0.0443  0.403    1.01     619.    1043.
## 6 u[6]      0.225  0.225 0.110 0.0972  0.0516  0.409    1.01     557.     909.
## 7 u[7]     -0.147 -0.147 0.110 0.0960 -0.321   0.0322   1.01     549.     933.
## 8 u[8]     -0.339 -0.335 0.143 0.140  -0.574  -0.108    1.01     974.    1584.
```

Bugged version for sigma

```
log_posterior_part1_bug <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance - beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}

mh_step_bug <- function(current, current_lp, proposal_sds, y, distance, buildingarea) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- log_posterior_part1_bug(proposal, y, distance, buildingarea)

  log_alpha <-  current_lp - proposal_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

mh_mixture_part1_bug <- function(y, distance, buildingarea,
                                 init,
                                 n_iter = 2000,
                                 proposal_sds_small = c(0.01, 0.0003, 0.000003, 0.01),
                                 proposal_sds_large = c(0.03, 0.0010, 0.000010, 0.03),
                                 mix_prob_small = 0.5) {

  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  current <- init
  current_lp <- log_posterior_part1_bug(current, y, distance, buildingarea)

  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
```

```
    use_small <- runif(1) < mix_prob_small
    sds <- if (use_small) proposal_sds_small else proposal_sds_large

    out <- mh_step_bug(
      current = current,
      current_lp = current_lp,
      proposal_sds = sds,
      y = y,
      distance = distance,
      buildingarea = buildingarea
    )

    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(
    samples = samples,
    acceptance_rate = mean(accepted)
  )
}

run_mh_from_forward_bug <- function(design_df, n_iter = 2000) {
  sim <- forward_part1_dataset(design_df)

  fit <- mh_mixture_part1_bug(
    y = sim$data$log_price,
    distance = sim$data$Distance,
    buildingarea = sim$data$BuildingArea,
    init = sim$theta,
    n_iter = n_iter
  )

  final_theta <- fit$samples[n_iter, ]

  list(
    initial_theta = sim$theta,
    final_theta = final_theta,
    acceptance_rate = fit$acceptance_rate
  )
}

set.seed(123)

n_rep <- 500
n_iter_check <- 2000

idx <- sample(seq_len(nrow(part1_design)), size = 200, replace = FALSE)
part1_design_small <- part1_design[idx, ]
```

```
initial_mat <- matrix(NA_real_, nrow = n_rep, ncol = 4)
final_mat   <- matrix(NA_real_, nrow = n_rep, ncol = 4)

colnames(initial_mat) <- c("beta0", "beta1", "beta2", "eta")
colnames(final_mat)   <- c("beta0", "beta1", "beta2", "eta")

for (m in 1:n_rep) {
  out <- run_mh_from_forward_bug(
    design_df = part1_design_small,
    n_iter = n_iter_check
  )

  initial_mat[m, ] <- out$initial_theta
  final_mat[m, ]   <- out$final_theta
}

initial_df <- as.data.frame(initial_mat)
final_df   <- as.data.frame(final_mat)

initial_df$sigma <- exp(initial_df$eta)
final_df$sigma   <- exp(final_df$eta)

ks_beta0 <- ks.test(jitter(initial_df$beta0, amount = 1e-10),
                    jitter(final_df$beta0,   amount = 1e-10))

ks_beta1 <- ks.test(jitter(initial_df$beta1, amount = 1e-10),
                    jitter(final_df$beta1,   amount = 1e-10))

ks_beta2 <- ks.test(jitter(initial_df$beta2, amount = 1e-10),
                    jitter(final_df$beta2,   amount = 1e-10))

ks_sigma <- ks.test(jitter(initial_df$sigma, amount = 1e-10),
                    jitter(final_df$sigma,   amount = 1e-10))

ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.02, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.012, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.004, p-value = 1
## alternative hypothesis: two-sided
```

```
ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 1, p-value = NA
## alternative hypothesis: two-sided
```

Bugged version for beta parameters:

```
# Beta0 bug
log_posterior_part1_bug_beta0 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  # B0 changed for log_prior
  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 20, sd = 0.25, log = TRUE) +   # BUG ON PURPOSE
    dnorm(beta1, mean = 0,  sd = 1,    log = TRUE) +
    dnorm(beta2, mean = 0,  sd = 1,    log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) +
    eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))
  log_prior + log_like
}

# Beta1
log_posterior_part1_bug_beta1 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  #   # B1 changed for log_prior
  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10,   log = TRUE) +
    dnorm(beta1, mean = 0.03, sd = 0.005,   log = TRUE) +   # BUG ON PURPOSE
    dnorm(beta2, mean = 0, sd = 1,    log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) +
    eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))
  log_prior + log_like
}

# Beta2
log_posterior_part1_bug_beta2 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
```

```
  sigma <- exp(eta)

  # BUG ON PURPOSE: wrong scaling of BuildingArea
  mu <- beta0 + beta1 * distance + beta2 * (10 * buildingarea)

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) +
    eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))
  log_prior + log_like
}
```

```
mh_step_with_lp <- function(current, current_lp, proposal_sds, y, distance, buildingare
a, lp_fun) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- lp_fun(proposal, y, distance, buildingarea)

  log_alpha <- proposal_lp - current_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

mh_mixture_with_lp <- function(y, distance, buildingarea,
                               init,
                               lp_fun,
                               n_iter = 2000,
                               proposal_sds_small = c(0.01, 0.0003, 0.000003, 0.01),
                               proposal_sds_large = c(0.03, 0.0010, 0.000010, 0.03),
                               mix_prob_small = 0.5) {

  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  current <- init
  current_lp <- lp_fun(current, y, distance, buildingarea)

  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
    use_small <- runif(1) < mix_prob_small
    sds <- if (use_small) proposal_sds_small else proposal_sds_large

    out <- mh_step_with_lp(
      current = current,
      current_lp = current_lp,
      proposal_sds = sds,
      y = y,
      distance = distance,
      buildingarea = buildingarea,
      lp_fun = lp_fun
    )

    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(
```

```
    samples = samples,
    acceptance_rate = mean(accepted)
  )
}

run_mh_from_forward_with_lp <- function(design_df, lp_fun, n_iter = 2000,
                                        proposal_sds_small = c(0.01, 0.0003, 0.000003,
0.01),
                                        proposal_sds_large = c(0.03, 0.0010, 0.000010,
0.03)) {
  sim <- forward_part1_dataset(design_df)

  fit <- mh_mixture_with_lp(
    y = sim$data$log_price,
    distance = sim$data$Distance,
    buildingarea = sim$data$BuildingArea,
    init = sim$theta,
    lp_fun = lp_fun,
    n_iter = n_iter,
    proposal_sds_small = proposal_sds_small,
    proposal_sds_large = proposal_sds_large
  )

  final_theta <- fit$samples[n_iter, ]

  list(
    initial_theta = sim$theta,
    final_theta = final_theta,
    acceptance_rate = fit$acceptance_rate
  )
}

run_invariance_bug_test <- function(lp_fun, design_df, n_rep = 500, n_iter_check = 2000,
seed = 123,
                                    proposal_sds_small = c(0.01, 0.0003, 0.000003, 0.0
1),
                                    proposal_sds_large = c(0.03, 0.0010, 0.000010, 0.0
3)) {
  set.seed(seed)

  idx <- sample(seq_len(nrow(design_df)), size = 200, replace = FALSE)
  design_small <- design_df[idx, ]

  initial_mat <- matrix(NA_real_, nrow = n_rep, ncol = 4)
  final_mat   <- matrix(NA_real_, nrow = n_rep, ncol = 4)

  colnames(initial_mat) <- c("beta0", "beta1", "beta2", "eta")
  colnames(final_mat)   <- c("beta0", "beta1", "beta2", "eta")

  for (m in 1:n_rep) {
    out <- run_mh_from_forward_with_lp(
      design_df = design_small,
```

```
      lp_fun = lp_fun,
      n_iter = n_iter_check,
      proposal_sds_small = proposal_sds_small,
      proposal_sds_large = proposal_sds_large
    )

    initial_mat[m, ] <- out$initial_theta
    final_mat[m, ]   <- out$final_theta
  }

  initial_df <- as.data.frame(initial_mat)
  final_df   <- as.data.frame(final_mat)

  initial_df$sigma <- exp(initial_df$eta)
  final_df$sigma   <- exp(final_df$eta)

  ks_beta0 <- ks.test(jitter(initial_df$beta0, amount = 1e-10),
                      jitter(final_df$beta0,   amount = 1e-10))

  ks_beta1 <- ks.test(jitter(initial_df$beta1, amount = 1e-10),
                      jitter(final_df$beta1,   amount = 1e-10))

  ks_beta2 <- ks.test(jitter(initial_df$beta2, amount = 1e-10),
                      jitter(final_df$beta2,   amount = 1e-10))

  ks_sigma <- ks.test(jitter(initial_df$sigma, amount = 1e-10),
                      jitter(final_df$sigma,   amount = 1e-10))

  list(
    ks_beta0 = ks_beta0,
    ks_beta1 = ks_beta1,
    ks_beta2 = ks_beta2,
    ks_sigma = ks_sigma,
    initial_df = initial_df,
    final_df = final_df
  )
}
```

Beta0 bug

```
out_beta0_bug <- run_invariance_bug_test(
  lp_fun = log_posterior_part1_bug_beta0,
  design_df = part1_design,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta0_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.464, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.04, p-value = 0.8186
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.004, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.748, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

Beta1 bug

```
out_beta1_bug <- run_invariance_bug_test(
  lp_fun = log_posterior_part1_bug_beta1,
  design_df = part1_design,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta1_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.024, p-value = 0.9987
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.2, p-value = 4.122e-09
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.004, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.604, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

Beta2 bug

```
out_beta2_bug_custom <- run_invariance_bug_test(
  lp_fun = log_posterior_part1_bug_beta2,
  design_df = part1_design,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123,
  proposal_sds_small = c(0.01, 0.0003, 0.0003, 0.01),
  proposal_sds_large = c(0.03, 0.0010, 0.0010, 0.03)
)

out_beta2_bug_custom$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.016, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug_custom$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.014, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug_custom$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.112, p-value = 0.003777
## alternative hypothesis: two-sided
```

```
out_beta2_bug_custom$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.802, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

Sensitivity check

setup

```
mod1_sens <- cmdstan_model("model1_sensitivity.stan")

stan_data_m1_base <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  x_dist = part1_data$Distance,
  x_area = part1_data$BuildingArea
)

prior_grid_m1 <- list(
  baseline = list(beta0_sd = 10, beta_sd = 1,   sigma_rate = 1),
  diffuse  = list(beta0_sd = 20, beta_sd = 2,   sigma_rate = 0.5),
  regular  = list(beta0_sd = 5,  beta_sd = 0.5, sigma_rate = 2)
)

fits_m1_sens <- list()

for (nm in names(prior_grid_m1)) {
  cat("Running prior setting:", nm, "\n")

  stan_data_now <- c(stan_data_m1_base, prior_grid_m1[[nm]])

  fits_m1_sens[[nm]] <- mod1_sens$sample(
    data = stan_data_now,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    seed = 123,
    refresh = 0,
    max_treedepth = 12,
    adapt_delta = 0.95
  )
}
```

```
## Running prior setting: baseline
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 96.6 seconds.
## Chain 2 finished in 129.5 seconds.
## Chain 4 finished in 137.0 seconds.
## Chain 3 finished in 232.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 148.8 seconds.
## Total execution time: 232.3 seconds.
##
## Running prior setting: diffuse
## Running MCMC with 4 parallel chains...
##
## Chain 4 finished in 128.4 seconds.
## Chain 1 finished in 141.4 seconds.
## Chain 2 finished in 220.5 seconds.
## Chain 3 finished in 226.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 179.3 seconds.
## Total execution time: 226.9 seconds.
##
## Running prior setting: regular
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 16.9 seconds.
## Chain 3 finished in 226.1 seconds.
## Chain 4 finished in 226.2 seconds.
## Chain 2 finished in 227.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 174.3 seconds.
## Total execution time: 227.8 seconds.
```

Diagnostics

```
fits_m1_sens$baseline$summary(c("beta0", "beta1", "beta2", "sigma"))
```

```
## # A tibble: 4 × 10
##   variable      mean    median       sd     mad       q5      q95  rhat ess_bulk
##   <chr>        <dbl>     <dbl>    <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.9      13.9       1.22e-2 1.24e-2  1.39e+1  1.39e+1  1.00    1196.
## 2 beta1    -0.0157   -0.0157    1.03e-3 1.03e-3 -1.74e-2 -1.40e-2  1.00    1467.
## 3 beta2     0.000114  0.000114  1.16e-5 1.16e-5  9.52e-5  1.33e-4  1.00    4084.
## 4 sigma     0.531     0.531     4.29e-3 4.31e-3  5.23e-1  5.38e-1  1.00    1439.
## # ℹ 1 more variable: ess_tail <dbl>
```

```
fits_m1_sens$diffuse$summary(c("beta0", "beta1", "beta2", "sigma"))
```

```
## # A tibble: 4 × 10
##   variable      mean    median       sd     mad       q5      q95  rhat ess_bulk
##   <chr>        <dbl>     <dbl>    <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.9      13.9       1.25e-2 1.25e-2  1.39e+1  1.39e+1  1.00    1199.
## 2 beta1    -0.0157   -0.0157    1.06e-3 1.05e-3 -1.75e-2 -1.40e-2  1.00    1408.
## 3 beta2     0.000114  0.000114  1.16e-5 1.17e-5  9.52e-5  1.34e-4  1.00    4167.
## 4 sigma     0.531     0.531     4.49e-3 4.34e-3  5.23e-1  5.38e-1  1.00    1287.
## # ℹ 1 more variable: ess_tail <dbl>
```

```
fits_m1_sens$regular$summary(c("beta0", "beta1", "beta2", "sigma"))
```

```
## # A tibble: 4 × 10
##   variable      mean    median       sd     mad       q5      q95  rhat ess_bulk
##   <chr>        <dbl>     <dbl>    <dbl>   <dbl>    <dbl>    <dbl> <dbl>    <dbl>
## 1 beta0    13.9      13.9       1.18e-2 1.19e-2  1.39e+1  1.39e+1 1.00     1338.
## 2 beta1    -0.0157   -0.0157    1.02e-3 1.01e-3 -1.75e-2 -1.40e-2 1.00     1444.
## 3 beta2     0.000114  0.000114  1.19e-5 1.17e-5  9.48e-5  1.34e-4 1.000    3493.
## 4 sigma     0.531     0.531     4.58e-3 4.55e-3  5.23e-1  5.39e-1 1.00     1216.
## # ℹ 1 more variable: ess_tail <dbl>
```

posterior summary table

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

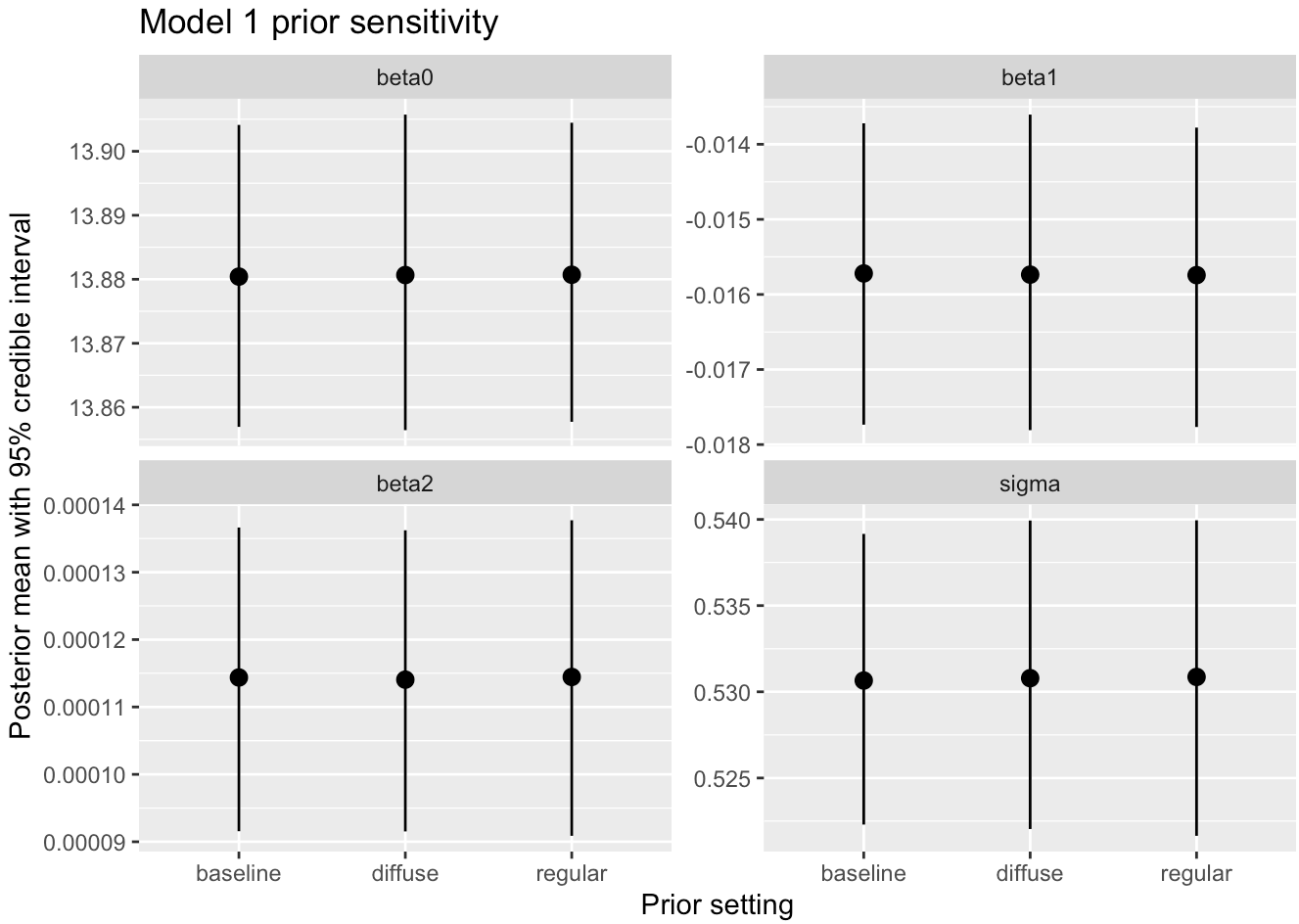
```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
summarize_fit_m1 <- function(fit, prior_name) {
  draws <- fit$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_
df")

  bind_rows(
    data.frame(
      prior = prior_name,
      parameter = "beta0",
      mean = mean(draws$beta0),
      median = median(draws$beta0),
      q2.5 = quantile(draws$beta0, 0.025),
      q97.5 = quantile(draws$beta0, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "beta1",
      mean = mean(draws$beta1),
      median = median(draws$beta1),
      q2.5 = quantile(draws$beta1, 0.025),
      q97.5 = quantile(draws$beta1, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "beta2",
      mean = mean(draws$beta2),
      median = median(draws$beta2),
      q2.5 = quantile(draws$beta2, 0.025),
      q97.5 = quantile(draws$beta2, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "sigma",
      mean = mean(draws$sigma),
      median = median(draws$sigma),
      q2.5 = quantile(draws$sigma, 0.025),
      q97.5 = quantile(draws$sigma, 0.975)
    )
  )
}

sens_table_m1 <- bind_rows(
  summarize_fit_m1(fits_m1_sens$baseline, "baseline"),
  summarize_fit_m1(fits_m1_sens$diffuse,  "diffuse"),
  summarize_fit_m1(fits_m1_sens$regular,  "regular")
)

sens_table_m1
```

```
##              prior parameter          mean        median          q2.5
## 2.5%...1  baseline     beta0 13.8804215105 13.8805920000  1.385694e+01
## 2.5%...2  baseline     beta1 -0.0157203028 -0.0157317565 -1.773517e-02
## 2.5%...3  baseline     beta2  0.0001143792  0.0001143558  9.155761e-05
## 2.5%...4  baseline     sigma  0.5306533828  0.5306452500  5.222967e-01
## 2.5%...5   diffuse     beta0 13.8806624027 13.8804185000  1.385642e+01
## 2.5%...6   diffuse     beta1 -0.0157361087 -0.0157402490 -1.780832e-02
## 2.5%...7   diffuse     beta2  0.0001140563  0.0001139163  9.152552e-05
## 2.5%...8   diffuse     sigma  0.5307907899  0.5307252550  5.220416e-01
## 2.5%...9   regular     beta0 13.8807082782 13.8803275000  1.385773e+01
## 2.5%...10  regular     beta1 -0.0157427857 -0.0157315325 -1.776607e-02
## 2.5%...11  regular     beta2  0.0001144584  0.0001144742  9.088208e-05
## 2.5%...12  regular     sigma  0.5308627262  0.5308014950  5.216470e-01
##                   q97.5
## 2.5%...1  13.9041283750
## 2.5%...2  -0.0137205829
## 2.5%...3   0.0001366368
## 2.5%...4   0.5391620630
## 2.5%...5  13.9057353750
## 2.5%...6  -0.0136042216
## 2.5%...7   0.0001362279
## 2.5%...8   0.5399363955
## 2.5%...9  13.9044684000
## 2.5%...10 -0.0137768394
## 2.5%...11  0.0001377192
## 2.5%...12  0.5399525363
```

Sensitivity comparison

```
library(ggplot2)
ggplot(sens_table_m1, aes(x = prior, y = mean, ymin = q2.5, ymax = q97.5)) +
  geom_pointrange() +
  facet_wrap(~ parameter, scales = "free_y") +
  labs(
    title = "Model 1 prior sensitivity",
    x = "Prior setting",
    y = "Posterior mean with 95% credible interval"
  )
```



Fit the model 1 and model 2

```
mod1 <- cmdstan_model("~/Model1.stan")
mod2 <- cmdstan_model("~/Part2_Model2_Hierarchical.stan")

part1_data <- part1_data %>%
  mutate(
    distance_z = as.numeric(scale(Distance)),
    buildingarea_z = as.numeric(scale(BuildingArea))
  )

stan_data_model1 <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z
)

part1_data$region_id <- as.integer(as.factor(part1_data$Regionname))

stan_data_model2 <- list(
  N = nrow(part1_data),
  R = length(unique(part1_data$region_id)),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z,
  region = part1_data$region_id
)
```

```
fit_m1 <- mod1$sample(
  data = stan_data_model1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.95,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 3 finished in 11.2 seconds.
## Chain 1 finished in 11.9 seconds.
## Chain 2 finished in 12.3 seconds.
## Chain 4 finished in 12.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 12.0 seconds.
## Total execution time: 12.8 seconds.
```

```
fit_m2 <- mod2$sample(
  data = stan_data_model2,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.99,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
```

```
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 1 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 1 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 1
```

```
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 2 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 2 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 2
```

```
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 3 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 3 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 3
```

```
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejecte
d because of the following issue:
```

```
## Chain 4 Exception: normal_lpdf: Location parameter[1] is inf, but must be finite! (in
'/var/folders/kk/bvvhj8cd0t1blhqn_fdpx0hr0000gn/T/RtmpXk3NcQ/model-553d286f51bb.stan', l
ine 36, column 2 to line 39, column 4)
```

```
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
types like covariance matrices, then the sampler is fine,
```

```
## Chain 4 but if this warning occurs often then your model may be either severely ill-c
onditioned or misspecified.
```

```
## Chain 4
```

```
## Chain 3 finished in 188.4 seconds.
## Chain 4 finished in 190.7 seconds.
## Chain 1 finished in 212.4 seconds.
## Chain 2 finished in 224.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 204.1 seconds.
## Total execution time: 224.8 seconds.
```

```
check_sampler_issues <- function(fit, treedepth_limit = 12) {
  sdg <- fit$sampler_diagnostics()

  data.frame(
    divergent = sum(sdg[, , "divergent__"]),
    hit_max_treedepth = sum(sdg[, , "treedepth__"] >= treedepth_limit),
    max_treedepth_seen = max(sdg[, , "treedepth__"])
  )
}

check_sampler_issues(fit_m1, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  3
```

```
check_sampler_issues(fit_m2, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  9
```

```
fit_m1$summary(c("beta0", "beta1", "beta2", "sigma")) %>%
  dplyr::select(variable, mean, sd, rhat, ess_bulk, ess_tail)
```

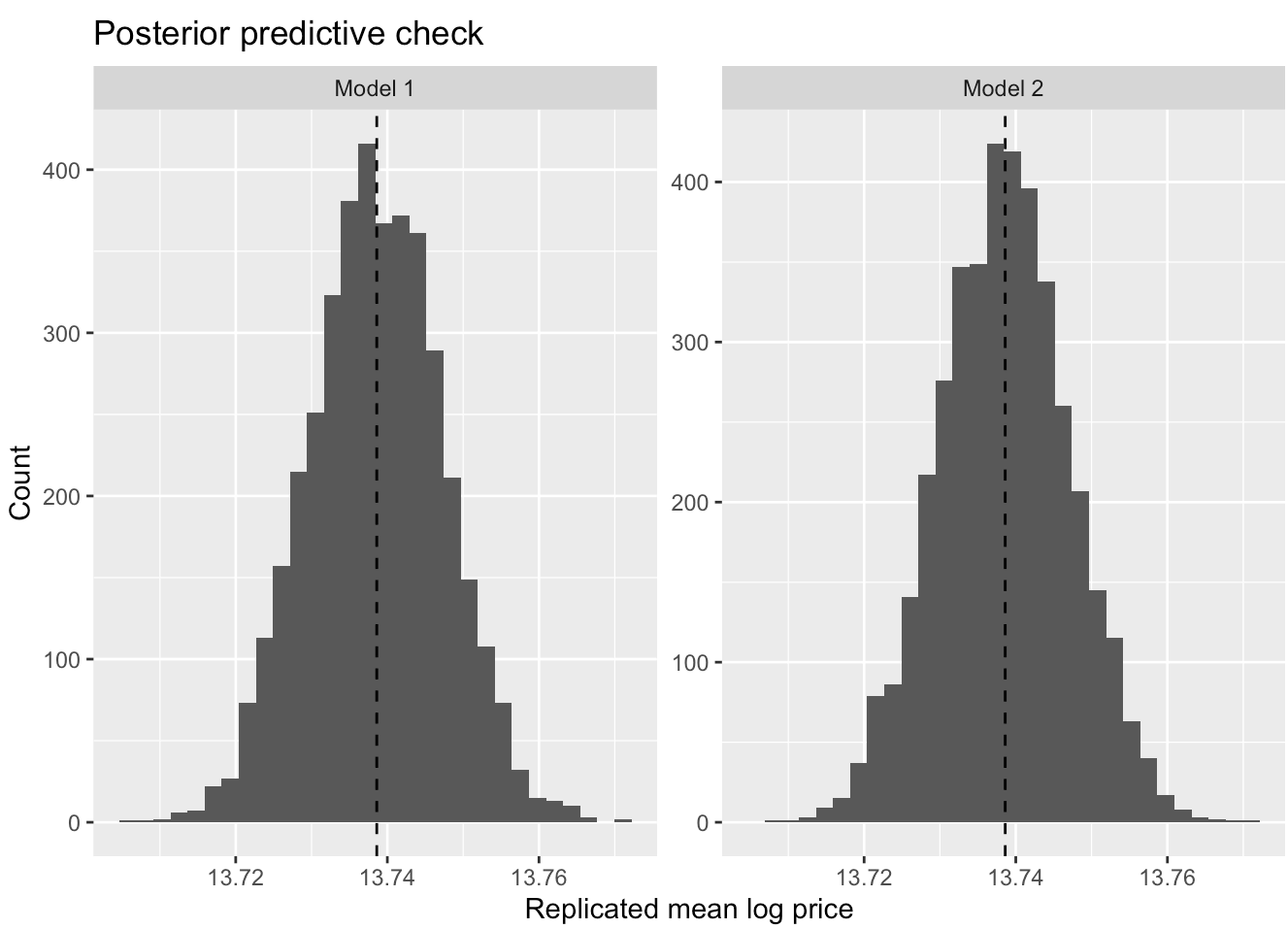
```
## # A tibble: 4 × 6
##   variable    mean      sd  rhat ess_bulk ess_tail
##   <chr>      <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    13.7    0.00626  1.00    5451.    2821.
## 2 beta1    -0.0947 0.00624  1.00    6043.    2666.
## 3 beta2     0.0621 0.00622  1.00    4698.    2783.
## 4 sigma     0.531  0.00434  1.00    4735.    2743.
```

```
fit_m2$summary(c("beta0", "beta1", "beta2", "tau", "sigma")) %>%
  dplyr::select(variable, mean, sd, rhat, ess_bulk, ess_tail)
```

```
## # A tibble: 5 × 6
##   variable    mean      sd  rhat ess_bulk ess_tail
##   <chr>      <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    13.7    0.109   1.01      805.    1079.
## 2 beta1    -0.123  0.00787 1.00     3318.    2839.
## 3 beta2     0.0618 0.00594 1.00     4017.    2545.
## 4 tau       0.276  0.0946  1.00     1004.    1647.
## 5 sigma     0.495  0.00411 1.000    3742.    2724.
```

posterior predictive check

```
yrep_m1 <- fit_m1$draws("y_rep", format = "matrix")
yrep_m2 <- fit_m2$draws("y_rep", format = "matrix")

ppc_df <- bind_rows(
  data.frame(
    model = "Model 1",
    y_rep_mean = rowMeans(yrep_m1)
  ),
  data.frame(
    model = "Model 2",
    y_rep_mean = rowMeans(yrep_m2)
  )
)

obs_mean <- mean(part1_data$log_price)

ggplot(ppc_df, aes(x = y_rep_mean)) +
  geom_histogram(bins = 30) +
  geom_vline(xintercept = obs_mean, linetype = 2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(
    title = "Posterior predictive check",
    x = "Replicated mean log price",
    y = "Count"
  )
```

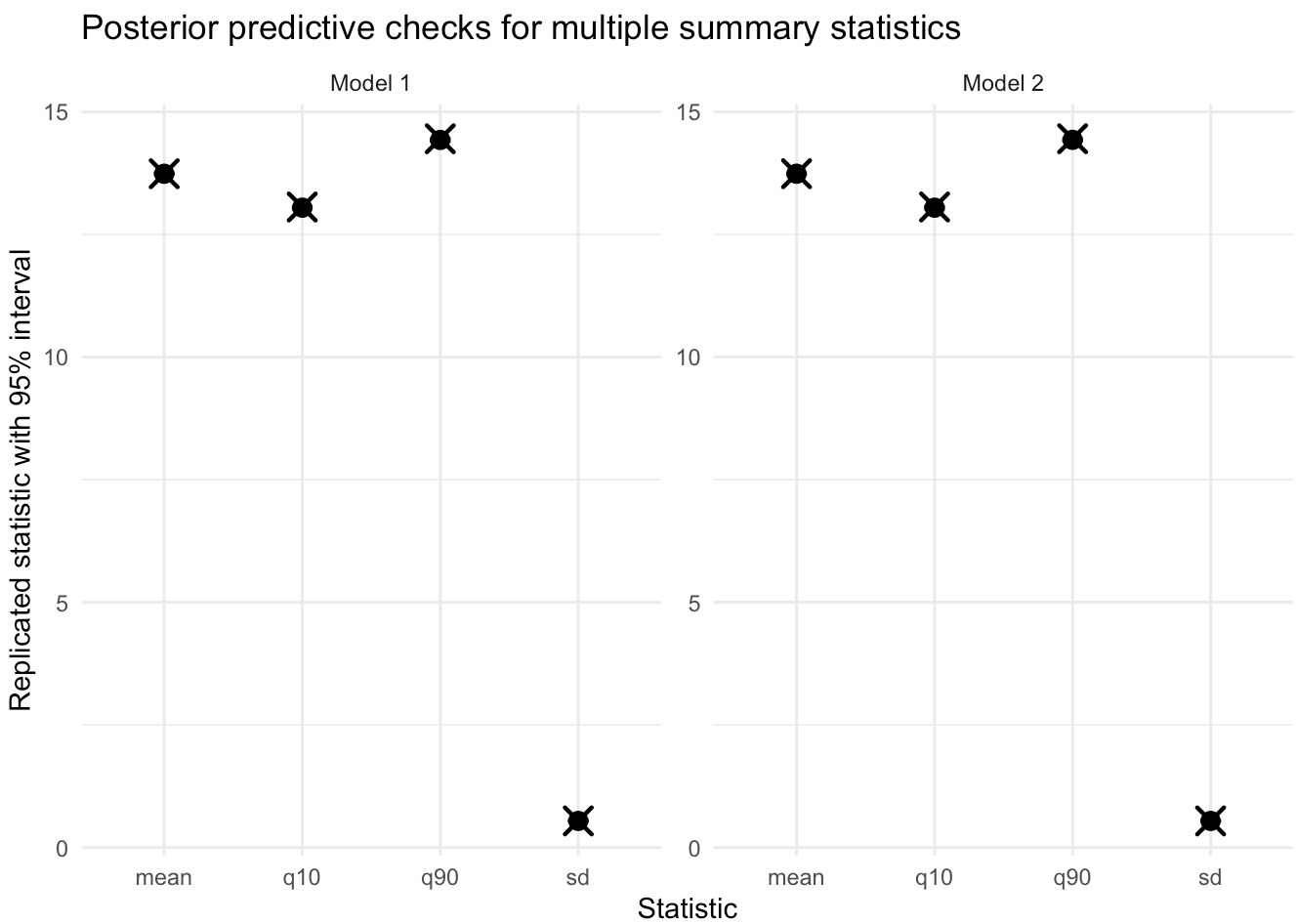


```
loglik_m1 <- fit_m1$draws("log_lik", format = "matrix")
loglik_m2 <- fit_m2$draws("log_lik", format = "matrix")

y_obs <- stan_data_model1$y

ppc_stats <- function(yrep, y_obs, model_name) {
  data.frame(
    model = model_name,
    stat = c("mean", "sd", "q10", "q90"),
    observed = c(
      mean(y_obs),
      sd(y_obs),
      quantile(y_obs, 0.10),
      quantile(y_obs, 0.90)
    ),
    rep_mean = c(
      mean(rowMeans(yrep)),
      mean(apply(yrep, 1, sd)),
      mean(apply(yrep, 1, quantile, probs = 0.10)),
      mean(apply(yrep, 1, quantile, probs = 0.90))
    ),
    lower = c(
      quantile(rowMeans(yrep), 0.025),
      quantile(apply(yrep, 1, sd), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.025)
    ),
    upper = c(
      quantile(rowMeans(yrep), 0.975),
      quantile(apply(yrep, 1, sd), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.975)
    )
  )
}

ppc_stats_m1 <- ppc_stats(yrep_m1, y_obs, "Model 1")
ppc_stats_m2 <- ppc_stats(yrep_m2, y_obs, "Model 2")

ppc_stats_all <- bind_rows(ppc_stats_m1, ppc_stats_m2)
ppc_stats_all
```

```
##     model stat   observed   rep_mean      lower      upper
## 1 Model 1 mean 13.7386074 13.7385243 13.7215386 13.7555082
## 2 Model 1   sd  0.5415036  0.5416035  0.5295896  0.5541168
## 3 Model 1  q10 13.0604880 13.0472719 13.0212414 13.0739663
## 4 Model 1  q90 14.4520874 14.4281008 14.4017240 14.4554547
## 5 Model 2 mean 13.7386074 13.7384670 13.7215861 13.7551988
## 6 Model 2   sd  0.5415036  0.5416756  0.5296656  0.5539859
## 7 Model 2  q10 13.0604880 13.0462788 13.0205078 13.0726710
## 8 Model 2  q90 14.4520874 14.4293222 14.4032802 14.4548169
```

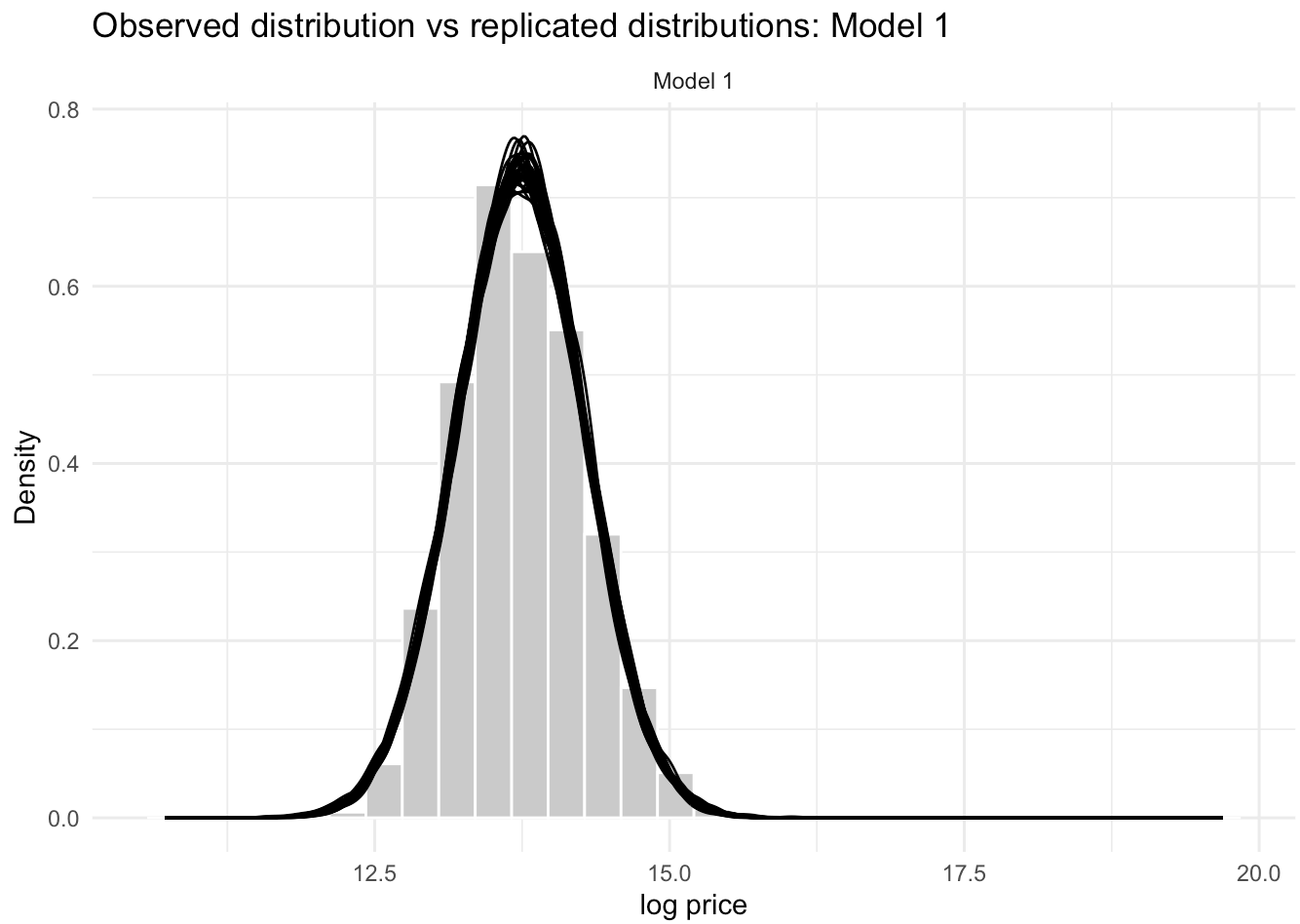
PCC estimates

```
ggplot(ppc_stats_all, aes(x = stat, y = rep_mean)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.15) +
  geom_point(aes(y = observed), shape = 4, size = 4, stroke = 1.2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(
    title = "Posterior predictive checks for multiple summary statistics",
    x = "Statistic",
    y = "Replicated statistic with 95% interval"
  ) +
  theme_minimal()
```



More check with observed histogram on replicated distributions

```
set.seed(123)

draw_ids_m1 <- sample(1:nrow(yrep_m1), 30)
draw_ids_m2 <- sample(1:nrow(yrep_m2), 30)

rep_df_m1 <- data.frame(
  value = as.vector(t(yrep_m1[draw_ids_m1, ])),
  draw = rep(1:30, each = ncol(yrep_m1)),
  model = "Model 1"
)

rep_df_m2 <- data.frame(
  value = as.vector(t(yrep_m2[draw_ids_m2, ])),
  draw = rep(1:30, each = ncol(yrep_m2)),
  model = "Model 2"
)

obs_df <- data.frame(value = y_obs)

ggplot() +
  geom_histogram(data = obs_df, aes(x = value, y = after_stat(density)),
                 bins = 30, fill = "grey80", color = "white") +
  geom_density(data = rep_df_m1, aes(x = value, group = draw),
               alpha = 0.15) +
  facet_wrap(~ model, scales = "free", ncol = 1, labeller = labeller(model = c("Model 1"
= "Model 1"))) +
  labs(
    title = "Observed distribution vs replicated distributions: Model 1",
    x = "log price",
    y = "Density"
  ) +
  theme_minimal()
```



LOO model comparsion

```
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.3.3
```

```
## This is loo version 2.8.0
```

```
## - Online documentation and vignettes at mc-stan.org/loo
```

```
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use
the 'cores' argument or set options(mc.cores = NUM_CORES) for an entire session.
```

```
##
## Attaching package: 'loo'
```

```
## The following object is masked from 'package:posterior':
##
##     gpdfit
```

```
loo_m1 <- loo(loglik_m1)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
loo_m2 <- loo(loglik_m2)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
loo_compare(loo_m1, loo_m2)
```

```
##        elpd_diff se_diff
## model2    0.0       0.0
## model1 -503.3      31.8
```

```
print(loo_m1)
```

```
##
## Computed from 4000 by 7113 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5613.4  81.2
## p_loo        41.8  39.1
## looic     11226.8 162.4
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                           Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7112  100.0%  3862
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_m2)
```

```
##
## Computed from 4000 by 7113 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5110.1  79.5
## p_loo        39.7  32.7
## looic     10220.1 159.0
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7112  100.0%  3503
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_compare(loo_m1, loo_m2))
```

```
##        elpd_diff se_diff
## model2    0.0       0.0
## model1 -503.3      31.8
```

Model 1

```
summ_m1 <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m1
```

```
##   variable        mean      median          sd          q5         q95     rhat
## 1    beta0 13.73866851 13.73853300 0.006261201 13.72850630 13.74889885 1.002203
## 2    beta1 -0.09467662 -0.09465115 0.006237670 -0.10514889 -0.08446412 1.002988
## 3    beta2  0.06209631  0.06211535 0.006217837  0.05186171  0.07231246 1.000960
## 4    sigma  0.53072058  0.53067008 0.004339532  0.52360574  0.53793203 1.000117
##   ess_bulk ess_tail
## 1 5450.812 2820.759
## 2 6042.643 2666.247
## 3 4697.784 2783.275
## 4 4735.304 2742.848
```

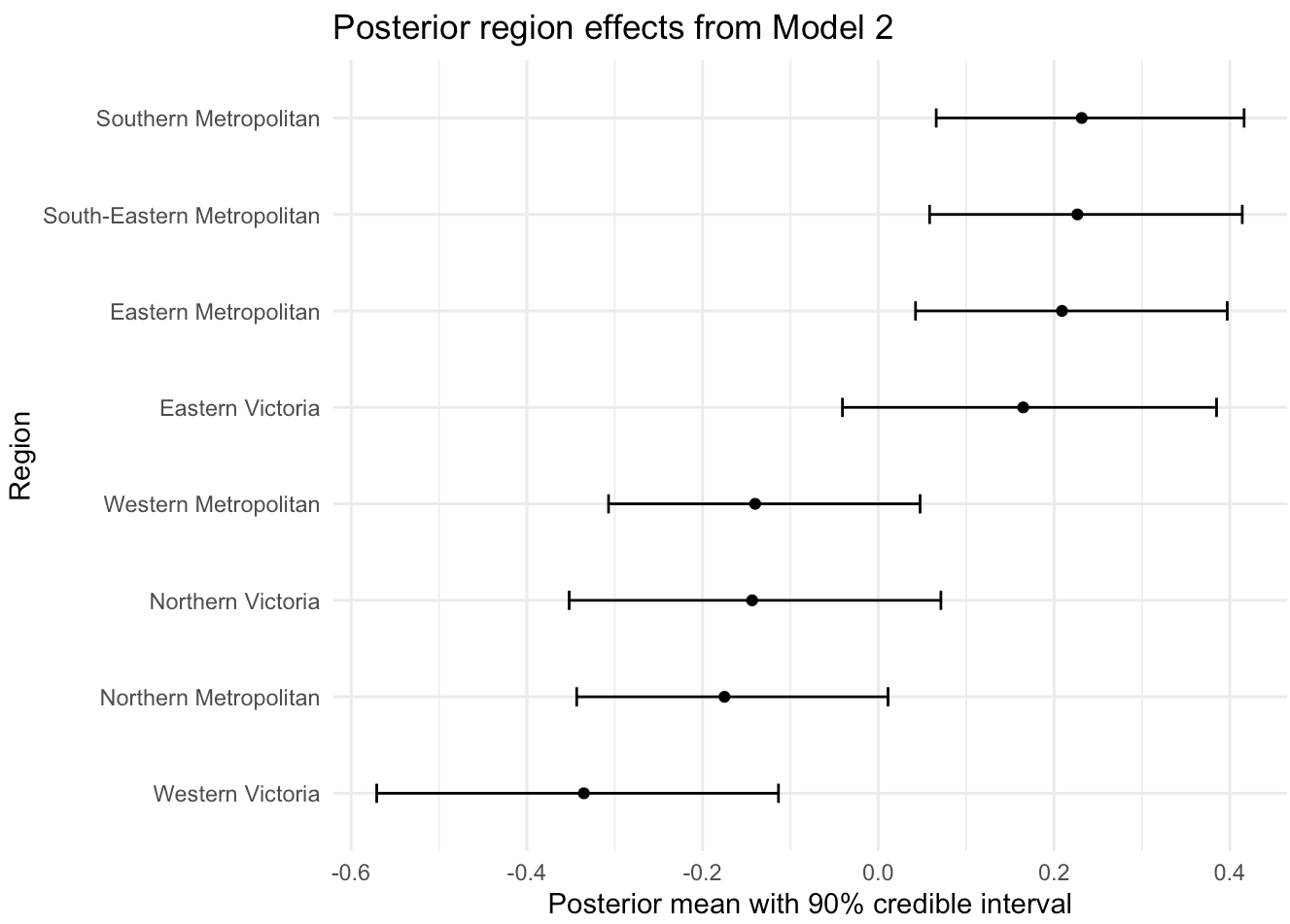
Model 2

```
summ_m2 <- fit_m2$summary(variables = c("beta0", "beta1", "beta2", "tau", "sigma")) %>%
  as.data.frame() %>%
  select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m2
```

```
##   variable        mean      median          sd          q5         q95     rhat
## 1    beta0 13.71643775 13.71965450 0.109130036 13.53168435 13.88238275 1.007401
## 2    beta1 -0.12279868 -0.12293477 0.007865449 -0.13547362 -0.10982977 1.000782
## 3    beta2  0.06176584  0.06163678 0.005938647  0.05208097  0.07162488 1.000294
## 4      tau  0.27570968  0.25649446 0.094633198  0.16360779  0.44928558 1.003048
## 5    sigma  0.49470527  0.49467696 0.004110390  0.48805359  0.50141664 0.999776
##    ess_bulk ess_tail
## 1  804.5328 1079.486
## 2 3317.9643 2838.624
## 3 4016.6960 2544.561
## 4 1003.8645 1647.137
## 5 3741.7147 2724.243
```

Regional effects from model 2

```
region_levels <- levels(as.factor(part1_data$Regionname))

u_summary <- fit_m2$summary(variables = "u") %>%
  as.data.frame() %>%
  mutate(region = region_levels) %>%
  select(region, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

u_summary
```

```
##                       region       mean     median        sd          q5
## 1       Eastern Metropolitan  0.2089652  0.2060361 0.1097475  0.04226725
## 2           Eastern Victoria  0.1648185  0.1605258 0.1321543 -0.04090005
## 3      Northern Metropolitan -0.1751213 -0.1795929 0.1101579 -0.34329465
## 4          Northern Victoria -0.1436700 -0.1417389 0.1323655 -0.35189370
## 5 South-Eastern Metropolitan  0.2265487  0.2215319 0.1115818  0.05834490
## 6      Southern Metropolitan  0.2314367  0.2278406 0.1099229  0.06587501
## 7       Western Metropolitan -0.1402073 -0.1442439 0.1096074 -0.30705496
## 8           Western Victoria -0.3351563 -0.3297296 0.1408306 -0.57090359
##           q95     rhat  ess_bulk ess_tail
## 1  0.39706315 1.007428  780.6531 1122.174
## 2  0.38491609 1.003660 1126.7507 1630.208
## 3  0.01101568 1.006889  827.8512 1084.384
## 4  0.07116957 1.005540 1270.9480 1423.392
## 5  0.41413004 1.006415  836.5745 1063.523
## 6  0.41622277 1.007252  835.2698 1059.549
## 7  0.04748189 1.007378  810.5395 1105.456
## 8 -0.11372327 1.004920 1383.8835 1791.197
```

```
ggplot(u_summary, aes(x = reorder(region, mean), y = mean)) +
  geom_point() +
  geom_errorbar(aes(ymin = q5, ymax = q95), width = 0.2) +
  coord_flip() +
  labs(
    title = "Posterior region effects from Model 2",
    x = "Region",
    y = "Posterior mean with 90% credible interval"
  ) +
  theme_minimal()
```